UNIVERSITÀ DEGLI STUDI DI BARI ALDO MORO



DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

TESI DI LAUREA

Sviluppo di Videogiochi

Adattività continua nei videogiochi: stima dello stato del giocatore mediante tecniche di anomaly detection su dati di input

Relatore:
Prof. Pierpaolo Basile

Laureando:
Raffaele Nacci

ANNO ACCADEMICO 2025/2026

Indice

Elenco delle figure **vii**

Elenco delle tabelle **ix**

1 Introduzione **1**

1.1 Contesto e motivazione 1
1.2 Il problema affrontato nella tesi 3
1.3 Domanda di ricerca e obiettivi 5
1.4 Struttura della tesi 6

2 Stato dell'arte **9**

2.1 Adattività e videogiochi: dall'approccio statico al DDA 9
2.1.1 Il problema della difficoltà statica 9
2.1.2 DDA: Dynamic Difficulty Adjustment basato su metriche di prestazione aggregate 10
2.1.3 DDA in videogiochi multiplayer: controllo e percezione da parte del giocatore 11
2.1.4 Affective computing e rilevamento fisiologico in tempo reale 13
2.1.5 Rilevamento dello stato emotivo di stress partendo dai soli input 14
2.1.6 Anomaly Detection unsupervised su baseline comportamentali 15
2.1.7 Approcci e tassonomie al *Player Modeling* 16
2.1.8 Generazione Procedurale di Contenuto (PCG) guidata dall'esperienza del giocatore 17
2.2 Problematiche ancora aperte 18

3 Metodologia **21**

3.1 Organizzazione concettuale del sistema 21
3.1.1 Panoramica dell'approccio utilizzato 21
3.1.2 Definizione operativa di stress comportamentale 22
3.2 Modellazione comportamentale del giocatore 23

3.2.1 Modello utilizzato in una sessione di gioco 23
3.2.2 Baseline comportamentale personalizzata 26
3.3 Rilevamento di anomalie e meccanica dello stress 27
3.3.1 Stima del punteggio di anomalia comportamentale 27
3.3.2 Integrazione temporale: accumulo dello stress 29
3.3.3 Adattamento del gameplay 30
3.4 Posizionamento della tesi rispetto allo stato dell'arte 31

4 Implementazione **33**
4.1 Panoramica dell'architettura 33
4.2 Acquisizione e definizione dell'input 35
4.2.1 Cattura e trasmissione dell'input 35
4.2.2 Classificazione dello stato di gioco 41
4.3 Elaborazione statistica e calibrazione lato server 44
4.3.1 Estrazione delle feature 44
4.3.2 Calibrazione e individuazione della baseline 46
4.4 Punteggio dell'anomalia e accumulazione dello stress 49
4.4.1 Calcolo del punteggio dell'anomalia 49
4.4.2 Accumulo dello stress nel tempo 52
4.5 Adattività del gameplay 54
4.5.1 Mondo psichedelico 54
4.6 Flusso dei dati e strumenti utilizzati 59
4.6.1 Flusso dei dati end-to-end 59
4.6.2 Strumenti e librerie usate 60

5 Sperimentazione **63**
5.1 Obiettivi e domande di validazione 63
5.2 Protocollo sperimentale 64
5.2.1 Protocollo calmo/frenetico 64
5.2.2 Raccolta dei dati e strutturazione del dataset 66
5.3 Metriche di valutazione 67
5.3.1 Veri positivi, falsi positivi e latenza di rilevamento 67
5.3.2 Il questionario GEQ 68
5.4 Risultati del rilevamento comportamentale 69
5.4.1 Evoluzione dello stress nel tempo: caso esemplificativo . 69
5.4.2 Risultati aggregati su più sessioni 70
5.5 Confronto tra i quattro modelli di anomaly detection 71
5.5.1 Risultati del confronto 71
5.5.2 Feature maggiormente responsabili del rilevamento . . . 73

5.6 Confronto tra la Versione adattiva (A) e quella non adattiva (B) 74
5.7 Discussione dei risultati e limiti 76

6 Conclusioni **79**
6.1 Riepilogo del lavoro svolto e valutazione degli obiettivi 79
6.2 Conclusioni 80
6.3 Sviluppi futuri 81

Bibliografia **85**

Elenco delle figure

4.1 Architettura generale del sistema: componenti lato client (Godot) e lato server (Python/FastAPI). Il ciclo cattura–richiesta–risposta–adattamento si ripete ogni $\Delta t = 5$ secondi. 34
4.2 Configurazione di `StressMonitor.gd` come autoload nel pannello Project Settings. 36
4.3 Assegnazione di un nemico boss al gruppo Godot `"Boss"`. 43
4.4 La barra dello stress comportamentale in un momento in cui è bassa all'interno di una sessione. 53
4.5 La barra dello stress comportamentale in un momento in cui è alta, quasi al limite all'interno di una sessione. 53
4.6 Modalità di gioco ordinaria. 58
4.7 Modalità di gioco `"Psichedelico"`. 58
4.8 Diagramma di sequenza del ciclo end-to-end. L'intero flusso si ripete a ogni finestra temporale ($\Delta t = 5$s) per l'intera durata della sessione. 59

5.1 Andamento del punteggio di anomalia comportamentale nel tempo per una sessione rappresentativa, con le fasi calma e frenetica del protocollo sperimentale annotate. 69
5.2 Confronto qualitativo tra i quattro modelli di anomaly detection per la sessione rappresentativa già presentata in Figura 5.1. . . . 72
5.3 Frequenza con cui ciascuna feature è risultata la principale responsabile del rilevamento, per la sessione rappresentativa. . . 73

Elenco delle tabelle

3.1 Le 22 feature comportamentali estratte da ciascuna finestra w_i. . 25

4.1 Mappatura dagli stati grezzi di `Player.gd` alle categorie di S. . 42
4.2 Strumenti e librerie utilizzati nello sviluppo del sistema. 61

5.1 Risultati del protocollo calmo/frenetico per le dieci sessioni del dataset. 70
5.2 Confronto tra i quattro modelli di anomaly detection, formulazione a 3σ, aggregato sulle dieci sessioni del dataset. . . 71
5.3 Confronto tra le sette sottoscale del GEQ Core Module, Versione A (adattiva) contro Versione B (non adattiva), $N = 10$. 74

Capitolo 1

Introduzione

1.1 Contesto e motivazione

L'idea di questa tesi parte da un interrogativo: É possibile creare un videogioco che, dinamicamente, riesce a cambiare gameplay in modo soggettivo, adattandosi ad ogni tipo di giocatore?
Dare una risposta a questa domanda non è stato semplice. Ad oggi esistono videogiochi che cercano di simulare una risposta adattiva nei confronti di un giocatore. Basti pensare a quei videogiochi che permettono di selezionare scelte diverse nel corso della storia, facendo intraprendere un percorso narrativo diverso da un altro. Esempi che riproducono quello appena descritto sono i seguenti titoli:

- la serie di Baldur's Gate
- la serie di Mass Effect
- The Witcher 3: Wild Hunt
- Detroit: Become Human
- la serie The Walking Dead (Telltale Games)
- ...

Tuttavia, questa tipologia si distacca dalla concezione di "*adattività*" che cercavo. Infatti, uno dei problemi relativi a quest'ultimo concetto è il **tipo** di *adattività* che viene proposta ai videogiocatori. Negli esempi citati abbiamo prodotti videoludici che fondano l'aspetto adattivo su parti specifiche del gioco: in base a una scelta del giocatore, a una missione piuttosto che un'altra, o a una risposta data a un NPC, ecc. il videogioco modifica il proprio andamento. Questi cambiamenti, però, sono valutati e predefiniti in fase di progettazione e sviluppo. Quindi, il problema mi ha portato ad una considerazione:

*«Questa tipologia di videogiochi guarda **cosa** il giocatore ha scelto, piuttosto che **come** si sta comportando nel momento in cui sta giocando.»*

Di conseguenza, l'adattamento dei videogiochi prima citati avviene a intervalli discreti, quindi non comporta alcun tipo di osservazione del comportamento di un giocatore al di fuori di quei momenti specifici.

Il lavoro che ho scelto di proporre in questa tesi si basa su un'idea che si colloca esattamente all'opposto. Piuttosto che osservare il giocatore in punti isolati del gioco, l'idea è proprio quella di tenerlo sotto osservazione durante tutta la sessione di gioco, in modo tale che il giocatore non arrivi mai a prendere una concreta ed esplicita decisione ma che sia il suo stesso modo di giocare a influenzare in maniera adattiva il gameplay a run-time, quindi in tempo reale. Ho così delineato in maniera strategica la differenza tra un gioco adattivo legato ad eventi discreti e un tipo di *adattività* continua e comportamentale. Oltre alla "semplice" idea di un videogioco adattivo, un'altra idea è quella che risponde ad uno dei tanti limiti dei videogiochi odierni, legato alla staticità della difficoltà. Un normale videogioco, infatti, è tipicamente organizzato in difficoltà crescenti selezionabili dal giocatore. Tuttavia, quest'ultimo durante una sessione di gioco non conserva mai lo stesso stato d'animo rispetto ad un momento precedente; inoltre, quando il giocatore affronta momenti diversi all'interno del videogioco stesso, spesso questi possono suscitare delle reazioni comportamentali ed emotive che potrebbero modificare il modo di giocare. Non è da escludere che il giocatore stesso, durante una sessione, possa passare da momenti di elevata padronanza a momenti di difficoltà o affaticamento; il tutto poi rimane invisibile al videogioco che ignora, per progettazione, questi cambiamenti. Questa discrepanza corrisponde proprio alla teoria del *flow* [5] secondo cui un'esperienza ottimale, per quanto riguarda il coinvolgimento, si raggiunge solamente quando la sfida percepita rimane in equilibrio dinamico con l'abilità del giocatore. Infatti, un livello di difficoltà bloccato a priori porta inevitabilmente a scivolare, ad un certo punto, verso la noia o, peggio ancora, verso la frustrazione.

La scelta di sviluppare un sistema con una meccanica adattiva è stata il modo per rispondere al problema di un gioco sviluppato in modo più statico. Un sistema concretamente adattivo si calibra sul comportamento di ogni singolo giocatore, regalando, così, un'esperienza di gioco più aderente allo stato del giocatore in ogni istante.

1.2 Il problema affrontato nella tesi

Concretamente il problema si può esemplificare con un semplice esempio: si immagini un giocatore mentre sta affrontando un nemico particolarmente forte o un boss. Il giocatore all'inizio dello scontro, interagendo tramite il controller, affronta una situazione tendenzialmente abbastanza stabile, quindi non particolarmente difficile. Questo significa che l'input è fluido con tempi di reazione regolari agli stimoli del videogioco.
Tuttavia, proseguendo con lo scontro la situazione diventa più concitata e frenetica, inducendo il giocatore ad avere una reazione comportamentale adeguata; il ritmo dello scontro porta il giocatore a interagire con il controller in modo più movimentato, con scatti dello stick più bruschi e imprevedibili.

Un normale osservatore umano si renderebbe conto che il giocatore si sta inevitabilmente agitando; proprio a partire dall'utilizzo del controller si noterebbe un cambiamento comportamentale riconducibile ad un aumento di stress, ansia, frustrazione o in generale un senso di agitazione.

Nella realtà, però, un videogioco non ha la stessa capacità dell'umano nell'accorgersi di una qualche sorta di cambiamento. L'unica cosa che sarebbe in grado di fare è continuare ad osservare, esclusivamente, le "grandezze oggettive e discrete" del videogioco, come ad esempio la vita, la stamina, o un qualche tipo di punteggio oppure, ancora, il numero di morti durante la sessione. In altre parole, un videogioco resterebbe "cieco" su **come** quelle grandezze sono state raggiunte.

La sfida di questo boss, giocata nelle stesse condizioni, ma da giocatori diversi, porterebbe alla conclusione oggettiva della fine dello scontro, senza però considerare in quali condizioni comportamentali si trovavano i giocatori mentre erano nel pieno dello scontro. Da questa considerazione, poc'anzi fatta, sintomo di una lacuna che affligge la gran parte dei videogiochi tradizionali, nasce la domanda:

«Si può dotare un videogioco della stessa capacità percettiva, che avrebbe un normale umano, rendendo tale capacità automatica e continua durante una sessione di gioco?»

Prima di rispondere a tale domanda sono sorti alcuni problemi:

- La strumentazione di rilevamento: in altre parole, esistono in letteratura e nella storia dei videogiochi, esempi di titoli in grado di rilevare stati comportamentali ed emotivi tramite vari sensori. Il problema resta che per realizzare un sistema del genere è necessario avere la strumentazione adatta, spesso costosa e difficile da configurare se non si conoscono tali strumenti, oltre ad altri ausili vari come ad esempio le webcam per il riconoscimento del volto o l'*eye-tracking*.

- La misurazione soggettiva: il comportamento di un giocatore non è un modello valido per tutti. Il modo di giocare più frenetico di un individuo poco esperto non risulta necessariamente tale se osservato in una persona più abituata ai videogiochi. Quindi non esiste una vera e propria soglia di distinzioni tra i due.

1.3 Domanda di ricerca e obiettivi

Per rispondere al quesito della sezione precedente, il lavoro da me svolto si è sviluppato in quattro macro obiettivi:

- **Costruzione di un sistema end-to-end** sviluppato in Godot 4 e integrato con un sistema di analisi comportamentale, scritto in Python, in grado di analizzare i pattern di input del giocatore e restituire in tempo reale un segnale che si riflette in una meccanica di gioco. L'output del sistema corrisponde ad un cambiamento del comportamento di gioco.
- **Definizione di un protocollo per la validazione** basato su due fasi di gioco iniziali, una più calma e l'altra più frenetica. Queste misurazioni mostrano l'affidabilità del sistema tramite metriche quantitative come TPR (True Positive Rate), FPR (False Positive Rate) e latenza di rilevamento.
- **Confronto tra quattro modelli di *Anomaly Detection***: gli input presi dal giocatore a run-time vengono analizzati da quattro modelli di apprendimento non supervisionato:
  - Isolation Forest
  - One-Class SVM
  - Local Outlier Factor
  - Elliptic Envelope
- **Validazione dell'esperienza di gioco** tramite la somministrazione del questionario GEQ (Game Experience Questionnaire), utilizzato per verificare e validare se i cambiamenti comportamentali rilevati dal sistema hanno una controparte coerente all'interno dei questionari, in modo da avere un feedback concreto direttamente da ogni singolo giocatore.

1.4 Struttura della tesi

La seguente tesi di laurea sviscera la soluzione trovata all'interrogativo posto all'inizio, ed è organizzata nel seguente modo:

- Capitolo 2: sarà presentata una rassegna della letteratura rilevante per questa tesi, organizzata su tre filoni principali. Il primo tratta degli approcci di *Dynamic Difficulty Adjustment*, sia nella loro forma classica basata su metriche di performance aggregate, sia nei contesti multiplayer. Il secondo riguarda il rilevamento dello stato del giocatore, distinguendo tra approcci di *affective computing* basati su sensori biometrici e approcci che stimano lo stato comportamentale a partire dai soli pattern di input, includendo un precedente sull'uso di *anomaly detection* non supervisionato su una baseline comportamentale personale, per quanto proveniente da un dominio applicativo estraneo ai videogiochi. Il terzo filone riguarda le tassonomie di *player modeling* e il framework di *Procedural Content Generation* guidata dall'esperienza (EDPCG). Il capitolo si chiude individuando le problematiche ancora aperte in letteratura, da cui emerge lo spazio non coperto in cui si colloca il lavoro proposto in questa tesi.

- Capitolo 3: sarà illustrata la metodologia da me proposta. Nello specifico, questa sezione non contiene parti relative agli strumenti implementativi. Nel capitolo sarà definito il modello basato su finestre temporali di 5 secondi sulle quali si sviluppa una sessione di gioco. Sarà spiegato il meccanismo di calibrazione delle diverse baseline comportamentali per ogni giocatore; inoltre, verrà descritta la funzione di stima del punteggio relativo all'anomaly detection, quindi alla dinamica di spostamento del fattore di "stress" all'interno del videogioco durante il gameplay. Alla fine di questo capitolo, l'approccio proposto in questa tesi sarà confrontato esplicitamente con le diverse soluzioni mostrate nel Capitolo 2.

- Capitolo 4: qui verrà descritta l'implementazione vera e propria del sistema, corredata da diversi diagrammi a blocchi relativi all'architettura complessiva del progetto. Saranno illustrati tutti i vari elementi descritti nel Capitolo 3 ma in modo più formale, descrivendo anche il flusso dei dati tra le due componenti: il videogioco in Godot 4 e il sistema in Python, nonché le librerie e gli strumenti utilizzati nello sviluppo di quest'ultimo.

- Capitolo 5: sarà descritta la sperimentazione condotta per la validazione del sistema, inclusa la descrizione di tutto il protocollo sperimentale, quindi la scelta di utilizzare parametri come calm/frenetic, impiegati per raccogliere diverse sessioni di gioco, e le metriche utilizzate: veri positivi, falsi positivi e latenza di rilevamento. Inoltre, sarà affrontato il confronto tra i diversi algoritmi di anomaly detection e i risultati dei questionari GEQ. Tutto ciò per descrivere la coerenza tra le variazioni comportamentali rilevate dal sistema e le varie esperienze soggettive di gioco.

- Capitolo 6: conterrà le conclusioni di questo lavoro. All'interno saranno descritte le valutazioni sugli obiettivi definiti da questa tesi e se questi ultimi sono stati raggiunti. Saranno inoltre discussi i possibili sviluppi futuri del sistema proposto.

Capitolo 2

Stato dell'arte

2.1 Adattività e videogiochi: dall'approccio statico al DDA

2.1.1 Il problema della difficoltà statica

É stato già discusso nel Capitolo 1 come la teoria formalizzata da Mihaly Csikszentmihalyi [5] dal nome:**Teoria del Flow**, abbia inquadrato l'equilibrio dinamico tra due elementi: la sfida percepita da un giocatore e l'abilità di quest'ultimo, come la condizione necessaria per avere un'esperienza coinvolgente ottimale. Tuttavia questa teoria è stata teorizzata in ambito psicologico generale, ma la sua applicabilità, per quanto pertinente al contesto di studio, ha richiesto una maggiore ricerca che andasse a interessare il mondo del game design.

In tal senso, il maggior apporto è stato portato dal modello *GameFlow* di Sweetser e Wyeth [16], le quali vanno a scomporre il *flow* teorizzato da Csikszentmihalyi e Csikszentmihalyi [5] in otto parti operative, non più una parola dal significato generico ma diversi specifici pezzi: la concentrazione, la sfida, l'abilità, il controllo, gli obiettivi chiari, il feedback, l'immersione, l'interazione sociale. Ognuna di queste parti è corredata da diverse domande che hanno la capacità di verificare la singola parte. In seguito, anche Cowley et al. [4] in "*Toward an understanding of flow in video games*" hanno descritto il concetto di *Flow* formalizzandolo oltre la semplice valutazione postuma. Infatti, loro proponevano un *Experience Fluctuation Model* che spiega come il *flow* non è solamente uno stato binario; anzi, corrisponde più ad una grandezza "fluttuate" nel tempo di una sessione di gioco, in funzione dei cambiamenti delle abilità o delle variazioni di sfida, quindi al fine di assecondare meccaniche adattive del gameplay per il singolo giocatore.

Da questa analisi emerge chiaramente quello che è un vero e proprio limite nei videogiochi con difficoltà statiche: se il flow di per sé una grandezza fluttuante, un livello di difficoltà fisso può, al più, intercettare un singolo istante della sessione, non seguirne l'andamento nel tempo. Come si vedrà nelle seguenti sezioni, la comunità scientifica cerca da tempo di risolvere o di superare questi limiti a partire da approcci consolidati e basati su metriche aggregate.

2.1.2 DDA: Dynamic Difficulty Adjustment basato su metriche di prestazione aggregate

La tecnica che prende il nome di ***Dynamic Difficulty Adjustment (DDA)*** è definita da Zohaib [21] come il processo, tale per cui, un videogioco ha la capacità di **adattarsi** al giocatore in modo da mantenerlo all'interno del, così detto da Mihaly Csikszentmihalyi, *Flow*. Questo grazie a processi di modifica automatica e sistemica in tempo reale delle sue caratteristiche come, gli scenari o i parametri di difficoltà del gioco. Zohaib descrive, attraverso una rassegna, come la tecnica del DDA sia un'area di ricerca in continuo sviluppo, seguendo approcci diversi, dall'utilizzo di intelligenze artificiali basate su reti neurali oppure reti bayesiane, ad approcci che puntano al coding e alla progettazione di codice atto a simulare l'adattività di un videogioco. Gli approcci in letteratura hanno un fattore comune, ovvero la difficoltà nella standardizzazione di un metodo. Questo emerge, con chiarezza, anche dal seguente confronto.

Storicamente uno dei sistemi DDA, particolarmente influente nella letteratura, è stato *Hamlet* [7]. Questo sistema DDA sviluppato per un FPS (First Person Shooter) costruito sul motore di gioco di *Half-Life*, tramite diverse funzioni implementate nel codice del gioco, estraeva una serie di parametri come: il contenuto dell'inventario, le morti durante la sessione, il danno subito da un avversario e non solo, per poi calcolare un valore probabilistico. Quando quest'ultimo raggiungeva una soglia critica il sistema interveniva sul videogioco modificando, ad esempio, il comportamento dei nemici (rendendoli più facili o meno da sconfiggere), oppure l'*economia* del gioco (posizionando più risorse, o più kit di cura). Tutto ciò, al fine di mantenere la salute del giocatore all'interno di una certa soglia, in modo da garantire che il giocatore continuasse a vivere all'interno dell'area del *Flow*.

Un altro valido esempio, opposto al precedente è quello proposto da Spronck, Sprinkhuizen-Kuyper e Postma [14] su una particolare tecnica che prende il nome di *Dynamic Scripting*, ovvero un metodo basato sull'esito degli scontri, con cui l'IA degli avversari sceglie una determinata tattica da un proprio registro di regole pesate. Nell'articolo vengono proposti tre meccanismi: la penalizzazione delle regole più efficaci (alle quali viene aumentato il *peso*), il contenimento dei pesi entro un certo intervallo prestabilito e l'eliminazione delle regole meno utilizzate. Questa meccanica nel gameplay permette di orientare il sistema verso uno stato di "*even game*". In altre parole, uno scontro rimane incerto fino alla fine della sua durata, invece che massimizzare la sola forza dell'IA come in una programmazione statica dell'avversario.

Tutti e due questi sistemi descritti, anche se operano su meccanismi differenti nella loro tecnica, condividono degli aspetti in comune: entrambi regolano la loro difficoltà sulla base di **metriche di performance aggregate**, il danno che un avversario infligge al personaggio di gioco, l'esito di scontri precedenti, oppure ancora gli elementi all'interno dell'inventario, i punti ferita e così via; partendo da una singola sessione oppure da un certo numero di sessioni, ma senza distinguere effettivamente gli stati comportamentali momentanei del giocatore. Infatti, per questo motivo risulta impossibile distinguere se un giocatore affronta con calma o con difficoltà un avversario.

2.1.3 DDA in videogiochi multiplayer: controllo e percezione da parte del giocatore

Nelle sezioni precedenti si è considerato il concetto di DDA in videogiochi a singolo giocatore; mentre, l'avversario è costituito dal sistema che può essere impostato liberamente dallo sviluppatore. Tuttavia, è doveroso citare anche la stessa teoria ma applicata a contesti multiplayer, infatti è importante comprendere come la teoria del DDA sia stata ampiamente discussa in letteratura, toccando diversi contesti videoludici. Nei videogiochi multiplayer il fattore di sfida non è rappresentato dagli avversari o dall'ambiente di gioco, ma dalla presenza di altri essere umani, questo introduce un ulteriore problema: l'intervento di una meccanica DDA che andrebbe a favorire un giocatore in difficoltà altererebbe necessariamente l'esperienza videoludica degli altri giocatori?

Da questo punto di vista, Baldwin, Johnson e Wyeth [1] hanno affrontato questo problema formulando il concetto di **istanza di Multiplayer DDA** (MDDA). L'idea alla base è quella di regolare le prestazioni di alcuni giocatori favorendo quelli in difficoltà e svantaggiando i partecipanti più abili, in questo modo secondo Baldwin, Johnson e Wyeth la differenza di sfida percepita dai vari partecipanti viene equilibrata in modo tale che anche i meno bravi abbiano la possibilità di "recuperare terreno". Nell'articolo si articolano sette diversi componenti progettuali, alcuni di essi sono: l'attivazione dell'istanza volontaria da parte di un giocatore oppure la sua assegnazione, la visibilità della presenza di questa istanza se resa pubblica o meno. Da questi sette punti hanno raccolto le valutazioni tramite un sondaggio somministrato a 150 partecipanti, distinguendoli in due categorie di prospettiva: gli *high performance perspective* (HPP) ovvero la prospettiva di chi gioca ad alte prestazioni e i *low performance perspective* (LPP) ovvero coloro che giocano a basse prestazioni.

Dopo la valutazione sono emersi risultati divergenti in base alla prospettiva: gli HPP preferiscono delle istanze di assistenza che siano strettamente dipendenti dall'abilità di un giocatore, magari attivabili con una specifica azione, al più una sola volta. Questo, in modo tale da non assecondare troppo il giocatore oscurando così il proprio reale merito; mentre, i giocatori LPP rivelano un maggior senso di tolleranza verso istanze che si possono utilizzare più volte, indipendentemente dalle abilità. Tuttavia, entrambe le posizioni convenivano sull'idea di avere istanze che fossero assegnate automaticamente dal sistema, piuttosto che averne di attivabili dai giocatori stessi, questo perché il rischio sarebbe quello di indurre in tentazione e abusare sistematicamente degli interventi; inoltre, entrambe le categorie attribuiscono importanza alla questione della visibilità degli interventi, ovvero ogni tipo di intervento dev'essere esplicito e visibile al giocatore in modo da capire anche a chi è stata dato l'aiuto.

Questo articolo introduce una diversa dimensione di studio che la Sezione 2.1.2 non affronta. Infatti, per quanto marginale con lo studio di questa tesi, questo argomento mostra come non sia importante, solamente, l'accuratezza della tecnica di rilevazione dello stato di un giocatore, in quanto l'attenzione su questo unico aspetto **non** corrisponde alla garanzia di un'esperienza di gioco equa e appagante, rispettando sempre il principio di *flow*. Il modo in cui un intervento viene attivato, reso noto (o meno) incide significativamente sulla sua accettazione, questa considerazione sarà di rilievo più avanti con la trattazione dei risultati e del questionario GEQ nel Capitolo 5.

2.1.4 Affective computing e rilevamento fisiologico in tempo reale

Esiste un altro versante di ricerca alternativo alla teoria del DDA basato su metriche di performance aggregate, che propone di ottenere una stima dello **stato affettivo** di un giocatore, che generalmente rientra in stati come l'ansia e l'eccitazione, per mezzo di sensori biometrici come: EEG, elettrodi per l'attività elettrodermica, ecc. L'utilizzo di questi sensori è incentrato sull'ottenere dati fisiologici per poi regolare la difficoltà di un gioco in tempo reale, in base alle stime rilevate. In questo settore, Rani, Sarkar e Liu [10] sono stati tra i primi a proporre un approccio, sviluppando un sistema che rileva e osserva i feedback fisiologici di un giocatore, mentre gioca ad un videogioco (nel caso dell'articolo, a una variante del gioco "*Pong*"), durante la sessione veniva tenuto sotto osservazione lo stato di stress, il quale veniva utilizzato per mantenere un livello di sfida e interesse ottimale. Questo stesso gruppo ha poi esteso il lavoro di ricerca nel seguente ambito mostrando ulteriori miglioramenti. Per quanto riguarda la loro sperimentazione, la ricerca ha mostrato che i partecipanti mostravano dei miglioramenti nelle prestazioni per molti dei sottoposti, dopo che il sistema valutava il feedback affettivo legato all'ansia.

Come per lo studio di Rani, Sarkar e Liu anche Chanel, Rebetez, Bétrancourt e Pun [3] applicano un metodo uguale ad un gioco differente da *Pong*, ovvero: "*Tetris*". In questo studio un campione di persone è stato fatto giocare a *Tetris*, il gioco è stato fatto provare in tre diversi livelli di difficoltà prestabiliti, durante le sessioni di gioco ai sottoposti sono stati rilevati i valori letti da un elettroencefalogramma (EEG), oltre che da vari rilevatori fisiologici periferici. Lo studio condotto da Chanel, Rebetez,B'etrancourt e Pun si componeva di due fasi: nella prima i sottoposti giocavano a *Tetris* mentre venivano monitorati i loro cambiamenti degli stati emotivi; infatti, questa stessa fase è risultata utile oltre la rilevazione, poiché ha permesso di verificare e validare l'applicabilità della *teoria del Flow* di Mihaly Csikszentmihalyi. Mentre nella seconda fase sono stati utilizzati gli stessi rilevamenti dei segnali fisiologici per classificare i vari stati emotivi, dimostrando così, per la prima volta in assoluto, la validità di un approccio come quello del DDA guidato al tempo stesso da un fattore psicologico come l'affetto. Infatti, lo studio ha permesso di formalizzare l'ipotesi che sia necessario modificare la difficoltà di un gioco in base alle risposte emotive e ai cambiamenti di stato emotivi di un giocatore.

Questi due sistemi appena visti hanno dei fattori in comune: primo fra tutti c'è la stima dello stato emotivo del giocatore (che in entrambi i casi verte di più su stati emotivi negativi come: ansia ed eccessivo eccitamento), in entrambi i casi lo stato del giocatore viene rilevato tramite sensori biometrici dedicati o generali, che corrisponde ad un approccio differente da quello scelto e sviluppato in questo lavoro, tuttavia rappresenta un metodo differente che storicamente ha prodotto diversi risultati promettenti e scientificamente dimostrati. Ciò nonostante, il vincolo strumentale, fattore comune nella letteratura legata all'affective computing applicato ai videogiochi, costituisce uno tra due "binari" lungo cui questo lavoro si distingue.

2.1.5 Rilevamento dello stato emotivo di stress partendo dai soli input

Un altro filone, diverso da quello precedente (Sezione 2.1.4), esplora uno scenario dove si cerca di stimare lo stato di un giocatore partendo solamente dagli schemi (pattern) di input, nonché l'interazione tra il giocatore e il videogioco, quest'ultimo passaggio senza l'utilizzo di sensori biometrici o di rilevamento generici. Wijayarathna e Lakshika [17] discutono su come questo campo di studio sia una "*questione ancora aperta*", realizzano, infatti, in una rassegna di diverse ricerche e tecniche per il rilevamento dello stress, sfruttando i videogiochi come mezzo a basso costo e di facile reperibilità. Wijayarathna e Lakshika fanno una distinzione tra approcci che si basano su misurazioni fisiologiche e altri che si basano su contesti domestici, quindi, privi di alcun tipo di strumentazione specifica e professionale. Quest'ultima categoria è esattamente quella dove si colloca questo lavoro.

Un altro contributo che si colloca in mezzo tra queste due linee di pensiero scientifico è quello condotto da Das, Bhattacharjee, Datta, Choudhury, Das e Pal [6], partendo dall'analisi dei tasti premuti durante una sessione di gioco che ha l'intento di indurre lo stress (integrando un test all'interno del gioco di *Tetris*). Questa parte del loro studio, seppur secondaria, utilizza sensori come elettroencefalogramma e risposta galvanica (per verificare i livelli di sudorazione di un giocatore) per costruire un riferimento di verità così da poter addestrare e validare un classificatore basato su tastiera. Das, Bhattacharjee, Datta, Choudhury, Das e Pal non hanno solo classificato il livello di stress in categorie discrete, ma sono riusciti a quantificare in modo continuo i valori di stress dei giocatori durante le sessioni di gioco.

Il lavoro degli autori evidenzia una diretta correlazione con informazioni sufficienti da rilevare uno stato di stress cognitivo, il tutto però coadiuvato da informazioni validate tramite risposta fisiologica dei giocatori.

L'ennesimo contributo su questo argomento risale già a circa vent'anni fa. Zimmermann et al. [20], discutono nella loro pubblicazione: *Affective Computing: A Rationale for Measuring Mood With Mouse and Keyboard* di come i dispositivi di input come la tastiera e il mouse, potessero essere in grado di funzionare come un "*canale*" abbastanza sufficiente per dedurre lo stato affettivo di una persona, senza ricorrere ad alcun tipo di strumentazione biometrica esterna. Nonostante il loro contributo in questo settore si collochi in un genere che **non** è quello videoludico, quindi che esula dallo studio di questa tesi; tuttavia, Zimmermann et al. [20] gettano le basi per quello stesso principio che negli anni è stato oggetto di studio anche in settori come quello videoludico, l'idea che il "***modo***" di interagire di una persona con un dispositivo hardware di input, possa inferire sul suo stato emotivo interno. Il loro lavoro si inserisce perfettamente all'interno di questa tesi perché a differenza dei sistemi presenti nella Sezione 2.1.4, il lavoro di questa tesi non utilizza sensori o non si occupa di addestrare un classificatore, quanto piuttosto tenta di rilevare tramite l'input di gioco lo stato comportamentale di un giocatore.

A differenza di questi contributi, il lavoro di questa tesi condivide l'assenza (o quasi assenza) di sensori biometrici, ma introduce un elemento che nessuno dei due contributi affronta, ovvero: la rilevazione tramite calibrazione, in un certo periodo, di una baseline comportamentale soggettiva per ogni soggetto.

2.1.6 Anomaly Detection unsupervised su baseline comportamentali

Fuori dall'ambito videoludico, già altri studiosi avevano pensato all'idea di costruire una baseline comportamentale e personale per ogni singola persona così da rilevare tutte le deviazioni tramite la stessa tecnica usata per questo lavoro di tesi: **Anomaly Detection Unsupervised**. Tra i primi ad occuparsi di questa tematica sono stati Sun, Versteeg, Boztas e Rao nell'ambito della sicurezza informatica. Anche se il loro lavoro esula il dominio di questa tesi, gli autori della ricerca [15] propongono l'applicazione (ed estensione) dell'algoritmo **Isolation Forest** [9] che in questo caso è capace di controllare variabili categoriche e numeriche. Il contesto di utilizzo, come già citato, riguarda il rilevamento di comportamenti anomali dei singoli utenti (sull'ordine delle centinaia) all'interno di un'azienda.

Per ogni utente è stato costruito un modello come riferimento, partendo dalle attività passate, in seguito sono state confrontate le azioni successive in modo tale che il modello potesse evidenziare attività anomale (come cambi di browser, log con email diverse, ecc.).

Tuttavia è doveroso segnalare come questo lavoro, nonostante sia pertinente nella forma, risulta pubblicato solamente come preprint, senza che sia mai transitato in una sede di revisione paritaria (*peer review*). Questa informazione viene qui dichiarata per onestà scientifica, non per sminuire il valore del contributo: pur non essendo transitato in una sede di revisione paritaria, questo lavoro resta infatti, per la ragione discussa di seguito, il precedente più vicino all'approccio proposto in questa tesi.
Questo metodo, infatti, resta uno tra quelli più vicini a quello applicato all'interno di questo lavoro di tesi, poiché come nell'esempio di Sun, Versteeg, Boztas e Rao, anche in questo è stata identificata una baseline individuale per ogni singolo giocatore.
Nonostante la validità del lavoro, il dominio scelto dagli autori dell'articolo resta distante da quello scelto per questa tesi, che invece verte sui videogiochi. A confermare la difficoltà dell'applicazione di un approccio come l'*anomaly detection* con il dominio videoludico, accostato all'utilizzo di pattern di input da controller, resta la scarsità di documenti in letteratura.

2.1.7 Approcci e tassonomie al *Player Modeling*

I vari sistemi che sono stati affrontati nelle sotto-sezioni precedenti, quindi dal DDA basato su metriche aggregate, al rilevamento di stati emotivi o affettivi tramite sensori biometrici o fisiologici e per finire alla stima comportamentale rilevata direttamente dagli input, ricadono in un'area di sviluppo che prende il nome di ***Player Modeling***, ovvero: la realizzazione di modelli computazionali dello stato, oppure dei tratti o, ancora, dell'esperienza ludica di un giocatore; il tutto, a partire dai dati che vengono raccolti durante una sessione di gioco. Yannakakis, Spronck, Loiacono e André [19] hanno realizzato un lavoro dove propongono una tassonomia sistematica, con lo scopo di collocare l'approccio utilizzato all'interno di questa tesi e altri approcci esistenti in letteratura. Gli autori spiegano **come** si può rilevare, modellare, prevedere ed esprimere caratteristiche di un giocatore umano, attraverso la creazione di un'IA e tecniche comportamentali. Ognuno di questi modelli si organizza su più elementi:

- **come si costruisce**: tramite un approccio *model-based* (partendo da una teoria già esistente, ad esempio in letteratura), oppure *model-free* (partendo da dati rilevati o esistenti, senza però dedurre nulla a priori)
- **cosa "entra" all'interno del modello** (input): tutto ciò che può essere utilizzato come dato di ingresso, come ad esempio dati di gameplay, dati oggettivi, biometrici, ecc. in modo da capire come si comporta nel tempo un giocatore
- **cosa "esce" dal modello** (output): ovvero valori scalari, dati ordinali, preferenze, ecc. (tramite clustering non supervisionato)

I sistemi di *affective computing* discussi in precedenza (Sezione 2.1.4) rientrano nella seconda categoria, poiché basati su segnali fisiologici; allo stesso modo, i sistemi basati su rilevamenti comportamentali anch'essi appartenenti alla seconda categoria, trattati in precedenza (Sezione 2.1.5), come il lavoro di questa tesi, sfruttano i dati di gameplay, senza l'utilizzo di strumenti biometrici.

Un'altra caratteristica importante riguarda la disponibilità di riferimenti con i quali validare il modello. Nel momento in cui viene trovata un'etichetta affidabile dello stato del giocatore, come ad esempio nel documento di Das et al. [6], oppure un'autovalutazione soggettiva, oppure una misura sensoriale o basata su gameplay, allora il modello del giocatore può essere addestrato tramite un approccio *supervisionato*. Tuttavia, se non è disponibile nessun tipo di riferimento in una sessione di gioco, allora diventa necessario utilizzare un approccio *non supervisionato*. Il lavoro proposto da questa tesi si colloca all'interno della tassonomia redatta da Yannakakis, Spronck, Loiacono e André e si confà ad un modello di *Player modeling* basato su dati oggettivi, ovvero: pattern di input.

2.1.8 Generazione Procedurale di Contenuto (PCG) guidata dall'esperienza del giocatore

Un altro esempio di *Player Modeling*, che riprende il concetto di adattività in un videogioco, finora discusso, è rappresentato dal seguente framework: "*Experience-Driven Procedural Content Generation*" (abbreviato in EDPCG), proposto da Yannakakis e Togelius [18].

Il lavoro di questa ricerca vede la formalizzazione di un ciclo, quest'ultimo si compone di tre elementi:

- Il primo corrisponde al gioco vero e proprio con il quale il giocatore interagisce.
- Il secondo elemento corrisponde ad un modello rappresentativo dell'esperienza videoludica del giocatore, i dati sono stati raccolti tramite i diversi metodi visti in precedenza, quindi: gameplay, dati fisiologici, soggettivi, ecc come già descritto nella sezione precedente 2.1.7.
- Il terzo elemento corrisponde ad un **generatore di contenuti**, quest'ultimo sfrutta il modello, del punto precedente, per creare nuovi contenuti come: livelli, nuove regole di gioco, meccaniche, avversari, scenari, ecc.

La principale connessione tra questo lavoro e lo studio di Yannakakis e Togelius [18] è un aspetto di carattere concettuale piuttosto che tecnico. All'interno di questo framework gli autori non descrivono un singolo intervento che agisce all'interno del videogioco (come ad esempio la regolazione della difficoltà), ma un intero ciclo, lo stesso descritto subito sopra. Questo ciclo ricorsivo prevede l'osservazione del giocatore, la costruzione di un modello e la conseguente modifica della struttura del videogioco. In altre parole, il lavoro presentato in questa tesi può essere considerato come un esempio più mirato e specifico rispetto a quello descritto in EDPCG.

2.2 Problematiche ancora aperte

Gli articoli relativi ai vari argomenti presenti all'interno di questo capitolo permettono di definire l'esistenza di tre strade presenti nella letteratura. Questi filoni evidenziano, sicuramente, un limite rispetto all'obiettivo di una meccanica adattiva e personalizzata pura e che risulti non invasiva durante una sessione. Tuttavia, sintetizzando lo scritto di questo capitolo abbiamo:

- **Sistemi di Dynamic Difficulty Adjustment classici** (Sezione 2.1): che regolano la difficoltà di un gioco partendo da metriche di performance aggregate come: danno dei punti ferita, fine di uno scontro, numero di vite perse, ecc. Come trattato nella Sezione 2.1.2 questo tipo di approccio non riesce a distinguere gli stati comportamentali di un giocatore in un preciso momento. Infatti, giocatori diversi possono anche raggiungere esiti uguali ma con modalità di difficoltà diverse fra loro e il sistema non sarebbe in grado di fare un distinguo.

Oltre a ciò, considerando la costruzione di questi sistemi, non riescono ad avere riferimenti sui singoli giocatori, ma lavorano in base a determinate soglie o politiche di adattamento (strutturate in fase di sviluppo) che siano valide durante l'intera sessione.

- **Sistemi di affective computing basati su sensori biometrici** (Sezione 2.1.4): che, in parte, superano il limite riuscendo a stimare lo stato affettivo di un giocatore in tempo reale all'interno di una sessione di gioco. Questo è reso possibile ad un costo: quello strumentale. Infatti, per arrivare a questo obiettivo questo sistema richiede che siano utilizzati sensori vari come: elettrodi, elettroencefalogrammi e altri, limitando la praticabilità del DDA fuori da un contesto di ricerca scientifica controllata.

- **Sistemi che stimano lo stato del giocatore da pattern di input** (Sezione 2.1.5): in questo caso vengono eliminate le problematiche legate al punto precedente, anche se tuttavia, resta indispensabile avere un riferimento fisiologico indipendente che permetta di addestrare un'IA (che possa essere classificatore, regressore o altro), come per il lavoro di Das et al.

Dalla rassegna presentata in questo capitolo, emerge come la ricerca abbia lasciato uno spazio scoperto all'interno della letteratura. Nessuno tra questi diversi approcci abbraccia simultaneamente quello di un'osservazione comportamentale continua e per nulla invasiva, con la calibrazione personale di ogni singolo giocatore campionata e stimata durante una sessione di gioco, con un terzo elemento che corrisponde ad un meccanismo di rilevamento che non richieda nessun tipo di sensore o rilevatore fisiologico.

Il lavoro presente all'interno di questa tesi si colloca esattamente in questo spazio non coperto dalla letteratura. Il posizionamento di questi diversi filoni è stato discusso in maniera più formale nella Sezione 3.4.

Capitolo 3

Metodologia

3.1 Organizzazione concettuale del sistema

3.1.1 Panoramica dell'approccio utilizzato

Nel Capitolo 1 si è definito il problema alla base di questa tesi. Infatti, nell'introduzione si è parlato di come poter osservare il comportamento di un singolo giocatore in una sessione di gioco, individuando a run-time delle deviazioni dal suo abituale modo di giocare, quindi individuare uno *standard* da utilizzare per permettere al videogioco di adattare dinamicamente il gameplay. All'interno di questo capitolo analizzo formalmente quale approccio ho scelto di adottare, tralasciando i dettagli implementativi discussi nel Capitolo 4.

Ogni sessione di gioco è il risultato dell'applicazione di quattro parti fondamentali, applicate una di seguito all'altra:

1. Frammentazione di una sessione di gioco in finestre temporali di cinque secondi, dove ogni finestra è stata rappresentata come un vettore di *features comportamentali*

2. Strutturazione di una *baseline comportamentale* soggettiva (una per ogni giocatore) che viene *definita* per ogni diversa categoria a seconda degli stati di gioco

3. Stima di un punteggio legato alla *anomalia comportamentale*, attraverso l'utilizzo di un modello non supervisionato di **anomaly detection**, che viene riconosciuta all'interno di una finestra rispetto alla corrispettiva baseline

4. Integrazione a run-time del punteggio, restituito dal modello, in un valore di "stress" accumulato all'interno del videogioco con la conseguente modifica del gameplay al raggiungimento della soglia critica dello stress

3.1.2 Definizione operativa di stress comportamentale

Prima di proseguire con la seguente sezione è doveroso precisare che il sistema proposto in questa tesi, per risolvere il quesito iniziale, non misura effettivamente lo stato di stress presente in un giocatore. In altre parole, tale rilevatore non rappresenta uno strumento clinico che misura clinicamente i livelli di stress di una persona mentre gioca ad un videogioco. Altrettanto importante è sottolineare che non rientra tra le pretese di questo lavoro presentare il sistema come uno strumento clinicamente riconosciuto, poiché un apparato del genere dovrebbe utilizzare sensori biometrici come: un rilevatore di frequenza cardiaca, un eye tracker, un insieme di elettrodi per il rilevamento dell'attività cerebrale, ecc. che comportano un costo esoso e delle competenze specialistiche, estranee all'ambito di questo lavoro. Quindi si sono utilizzati come unici input quelli catturati dal controller impiegato per giocare la sessione, dati che sono stati analizzati ai fini di riconoscere eventuali pattern.

Quindi, operativamente parlando lo *stress comportamentale* rappresenta una misura della **deviazione statistica del comportamento di input del singolo giocatore rispetto alla sua baseline**, il tutto all'interno della stessa sessione di gioco. Quest'ultimo aspetto corrisponde, infatti, ad una scelta mirata importante: il sistema non restituisce una valutazione assoluta sul giocatore, dicendo se quest'ultimo è stressato o meno, riferisce in che momento di gioco il suo modo di giocare differisce dal suo *standard*. Questa caratteristica, che sarà discussa più avanti nella Sezione 3.2.2 è una parte centrale nella soluzione suggerita in questa tesi.

Parallelamente alla soluzione proposta, un'altra validazione dello scostamento comportamentale presente nella versione del gioco adattiva, rispetto all'esperienza di gioco priva della modifica è discussa nel Capitolo 5.

3.2 Modellazione comportamentale del giocatore

3.2.1 Modello utilizzato in una sessione di gioco

Una sessione di gioco è formata da una sequenza di intervalli temporali non sovrapposti di una certa durata fissa Δt (cinque secondi nel caso di studio della tesi). Considerando la seguente formula:

$$W = (w_1, w_2, \ldots, w_n), \qquad w_i = [t_i,\, t_i + \Delta t) \tag{3.1}$$

Come si può osservare, a ciascuna finestra w_i viene associato uno stato di gioco $\sigma(w_i) \in S$, dove S rappresenta l'insieme delle categorie di gioco importanti per la calibrazione:

$$S = \{\,\text{idle},\ \text{traversal},\ \text{combat_regular},\ \text{combat_boss}\,\} \tag{3.2}$$

Oltre a queste categorie, all'interno dei log, un numero esiguo di finestre (all'incirca da 2 a 7 per ogni sessione) sono state associate ad uno stato di gioco che non si trova in S, questo stato nasce quando il sistema non riesce a categorizzare una finestra. Queste finestre non vengono trattate come le altre, ma seguono lo stesso meccanismo di fallback previsto per un qualsiasi altro stato poco campionato, finendo sempre per essere attribuite alla baseline globale, proprio perché rimangono al di sotto della soglia minima k_{min}. Nonostante ciò, nel momento in cui le finestre raggiungono la soglia k_{min} all'interno di una sessione, allora anch'esse riceverebbero una loro baseline come qualsiasi altro stato.

La scelta metodologica di distinguere le categorie: `combat_regular` e `combat_boss` nasce dalla volontà di voler distinguere gli scontri con diverse tipologie di avversari. In ordine, nella prima categoria ricadono quei nemici il cui combattimento ha una bassa intensità per costruzione in termini di level design; mentre, la seconda categoria corrisponde a quella dove gli scontri hanno un'intensità più alta ed è più difficile arrivare alla fine del combattimento.

La scelta di distinguere le due categorie è una conseguenza del fatto che, avere una sola categoria che identificasse il combattimento in generale, avrebbe portato il sistema a definire una qualsiasi tipologia di scontro in modo "*diluito*", quindi meno sensibile alle reali deviazioni comportamentali, argomento spiegato meglio in Sezione 3.2.2.

Per ognuna delle finestre w_i si definisce una funzione di estrazione delle caratteristiche $\varphi : w_i \mapsto x_i \in \mathbb{R}^d$, dove d è pari a 22 nella configurazione scelta.

Gli elementi di x_i sono raggruppati in quattro diverse categorie scelte in fase di progettazione, ciascuna rappresenta un aspetto comportamentale della *frenesia* dell'input del giocatore:

- **feature di frequenza**: numero di pressioni e rilasci di tasti nella finestra, numero di tasti diversi coinvolti e frequenza di pressione;
- **feature distribuzionali**: entropia nella scelta dei tasti premuti e tasso di diversità delle pressioni, come misura di quanto il giocatore concentri l'input su un singolo tasto anziché variarlo;
- **feature di dinamica temporale**: intervallo medio e variabilità tra pressioni consecutive, *burstiness* (ovvero la tendenza a concentrarsi in raffiche ravvicinate di pressioni), ed energia, ampiezza e variabilità del movimento dello stick analogico;
- **feature differenziali**: variazione delle grandezze precedenti rispetto alla finestra immediatamente precedente e media mobile calcolata sulle ultime finestre, per catturare non solo lo stato istantaneo ma anche la sua tendenza.

Di seguito la tabella 3.1 che riporta l'elenco esaustivo di tutte e 22 le feature:

Categoria	Feature	Descrizione
Frequenza	key_press_count	numero di pressioni di tasti nella finestra
	key_release_count	numero di rilasci di tasti nella finestra
	unique_keys	numero di tasti diversi premuti nella finestra
	key_frequency	frequenza di pressione dei tasti nella finestra
Distribuzionali	key_entropy	entropia della distribuzione dei tasti premuti
	key_diversity_rate	tasso di diversità delle pressioni (basso = pressioni concentrate su un solo tasto)
Dinamica temporale	avg_inter_press	intervallo medio tra pressioni consecutive
	std_inter_press	irregolarità dell'intervallo tra pressioni
	burstiness	tendenza a premere i tasti "a raffica"
	stick_mean	posizione media dello stick analogico
	stick_std	variabilità di oscillazione dello stick
	stick_max	inclinazione massima dello stick nella finestra
	stick_count	numero di rilevazioni dello stick nella finestra
	stick_range	escursione tra valore minimo e massimo dello stick
	movement_energy	energia complessiva del movimento dello stick
	sign_changes	numero di cambi di direzione dello stick
	action_intensity	somma di pressioni di tasti e rilevazioni dello stick
Differenziali	delta_key_press	variazione del numero di pressioni rispetto alla finestra precedente
	delta_key_freq	variazione della frequenza di pressione rispetto alla finestra precedente
	delta_stick_std	variazione della variabilità dello stick rispetto alla finestra precedente
	rolling_key_freq	media della frequenza di pressione sulle ultime finestre
	rolling_stick_variability	media della variabilità dello stick sulle ultime finestre

Tabella 3.1: Le 22 feature comportamentali estratte da ciascuna finestra w_i.

3.2.2 Baseline comportamentale personalizzata

Il cuore metodologico del sistema è la definizione di una baseline che sia *personale* e, al tempo stesso, *contestuale*, quindi specifica per ognuno dei giocatori e per ognuno degli stati, prima citati ($s \in S$).
La fase di calibrazione iniziale per lo stato s è definita come l'insieme delle feature, in forma vettoriale, che vengono osservate in quel determinato stato durante la fase iniziale della sessione di gioco, che ha la durata di circa sessanta secondi:

$$C_s = \{ x_i : \sigma(w_i) = s \} \tag{3.3}$$

Per costruire la baseline per uno stato dedicato $s \in S$, ho utilizzato la seguente formula: $B_s = (\mu_s, \sigma_s)$ con $\mu_s, \sigma_s \in \mathbb{R}^d$. Quest'ultima è definita dalle statistiche di calibrazione, ovvero: media e deviazione standard per ogni feature, direttamente derivate da C_s. Per quegli stati che invece non riescono a raccogliere abbastanza finestre ($|C_s| < k_{min}$), allora si costruisce una baseline globale B_0, la quale viene costruita sulla base delle prime finestre osservate in sessione, indipendentemente dallo stato che veniva riconosciuto.

Quindi, formalmente, la baseline concretamente utilizzata per ogni finestra w_i corrisponde a:

$$B_{\sigma(w_i)} = \begin{cases} B_s, & \text{se } B_s \text{ è stata stimata (con } s = \sigma(w_i)) \\ B_0, & \text{altrimenti} \end{cases} \tag{3.4}$$

La baseline B_s viene realizzata nel momento in cui C_s raggiunge la soglia minima k_{min}, dopodiché B_s continuerà ad esistere per ogni finestra successiva dello stato s, indipendentemente dalla continua crescita di C_s. Quindi, la condizione dell'equazione 3.4 fa sì che esista sempre una baseline già calcolata, piuttosto che verificare la cardinalità di ogni finestra.

Quando viene raggiunto il minimo di finestre per-stato allora la baseline viene *congelata*, ovvero: da quel momento in poi non viene più aggiornata con le finestre per-stato successive. Questa caratteristica è stata, volutamente, presa come scelta di design nonostante non sia l'unica. Infatti l'altra possibilità sarebbe stata quella di avere una *baseline mobile* che venisse aggiornata continuamente. Quest'ultima modalità è stata esclusa poiché se un giocatore avesse mantenuto uno stato di *stress comportamentale* per un tempo prolungato, allora la baseline avrebbe potuto assorbire tale rilevazione, "normalizzando" quel tipo di comportamento, perdendo, però, la capacità di rilevare nuovamente lo stress.

Infatti, il compromesso è stato quello di accettare un'adattività minore della baseline, a fronte di una maggior sensibilità in termini di rilevamento dello *stress comportamentale*, che rimaneva la proprietà principale ai fini di questa tesi.

3.3 Rilevamento di anomalie e meccanica dello stress

3.3.1 Stima del punteggio di anomalia comportamentale

Considerando il vettore delle feature x_i estrapolato da una finestra w_i, il sistema calcola un punteggio di "**anomalia**" grezzo, grazie al modello di ***unsupervised anomaly detection*** – nel confronto sperimentale del Capitolo 5 vengono considerati quattro modelli di questa famiglia: **Isolation Forest** [9], **One-Class SVM** [12], **Local Outlier Factor** [2] ed Elliptic Envelope, basato sull'algoritmo di **Minimum Covariance Determinant** [11]. Inoltre, una rassegna più ampia sui vari algoritmi della famiglia della One-Class Classification a cui appartengono questi approcci è fornita da [13]. Di seguito, il punteggio grezzo si esprime come:

$$a_i = M_{\sigma(w_i)}(x_i) \tag{3.5}$$

Dove $M_{\sigma(w_i)}$ è il modello di *Isolation Forest* addestrato sulla baseline attiva per quella finestra (per-stato o globale, Sezione 3.2.2). É stato scelto questo modello per le sue diverse proprietà: non richiede che i dati abbiano forme della distribuzione particolari, funziona bene anche con poche osservazioni di calibrazione rispetto al vettore delle feature e ha un costo computazionale basso considerando il suo funzionamento a run-time.

All'inizio il sistema parte con il vettore di 22 feature da quale estrapola un punteggio grezzo: a_i. Tuttavia, questo punteggio non è interpretabile, poiché non permette di capire se l'anomalia farà crescere o decrescere la barra dello stress comportamentale. Quindi, il punteggio viene trasformato in due diverse grandezze, che servono entrambe al fine di formare il punteggio finale: una *magnitudo*, che indica quanto vale, in termini di intensità, l'anomalia della finestra, e una *direzione* che indica dove deve muoversi lo scostamento della barra, quindi se il comportamento del giocatore è più intenso o più pacato rispetto al suo standard.

Nello specifico la *magnitudo* si ottiene normalizzando a_i (ovvero il punteggio grezzo calcolato su una nuova finestra dall'isolation forest) con il riferimento alla distribuzione dei punteggi registrati durante la calibrazione (ovvero media ν e deviazione standard τ di a calcolati sempre dall'isolation forest). A questo punto avviene un "taglio" all'interno di un certo limite e infine, la divisione per 2:

$$\hat{m}_i = \frac{1}{2}\,\mathrm{clip}\left(\frac{a_i - \nu_{\sigma(w_i)}}{\tau_{\sigma(w_i)}},\ 0,\ 2\right) \in [0,1] \tag{3.6}$$

Per quanto riguarda la *direzione*, questa viene calcolata guardando il valore delle singole feature rispetto alla nuova finestra: $x_{i,j}$. Per ognuna di queste si calcola lo scostamento rispetto alla media e all'oscillazione catturate durante la calibrazione. A questo punto si effettua la media complessiva: se il valore calcolato è positivo allora viene preso direttamente quello, altrimenti viene attenuato tramite un moltiplicatore, pur restando sempre un valore non negativo:

$$d_i = \frac{1}{d}\sum_{j=1}^{d}\frac{x_{i,j} - \mu_{\sigma(w_i),j}}{\sigma_{\sigma(w_i),j}} = \frac{1}{d}\sum_{j=1}^{d} zscore_j \tag{3.7}$$

Inoltre, lo $zscore_j$ (ovvero lo $z_{i,j}$) per feature $z_{i,j} = (x_{i,j} - \mu_{\sigma(w_i),j})/\sigma_{\sigma(w_i),j}$ riesce anche ad individuare la singola feature che ha, principalmente, causato la deviazione. Infatti, $\arg\max_j |z_{i,j}|$ è l'informazione utile a mostrare il perché una particolare finestra viene segnalata come anomala.

Il punteggio ultimo che influisce sul gameplay, spostando la barra dello stress comportamentale combina i due valori di magnitudo e direzione applicando poi un fattore di asimmetria $\lambda \in (0,1)$. Questo perché, come accennato prima, quando la direzione del punteggio finale risulta negativa, evidenziando un comportamento più pacato rispetto allo standard, non azzera completamente il punteggio, ma la barra rimane visivamente reattiva anche in fase decrescente:

$$\hat{a}_i = \begin{cases} \hat{m}_i, & d_i > 0 \\ \lambda \cdot \hat{m}_i, & d_i \leq 0 \end{cases} \tag{3.8}$$

Resta importante evidenziare che questa struttura: di saturazione e asimmetria direzionale della barra dello stress, sono elementi che caratterizzano solamente l'esperienza videoludica durante il gameplay, stesso insieme di caratteristiche che sostengono le metriche di valutazione, come si vedrà nel Capitolo 5, quindi veri positivi, falsi positivi e latenza.

Inoltre, le differenze sperimentali analizzate tra i quattro diversi modelli di *Anomaly Detection* sono, per scelta, più semplici e uniformi; infatti, per ogni modello scelto il punteggio grezzo, che poi viene normalizzato rispetto alla distribuzione della calibrazione nella fase iniziale di gioco, non ha una saturazione di 2σ, ma ne ha una di 3σ, senza tenere in considerazione asimmetria direzionale data dal moltiplicatore, e prendendo in esame solamente la baseline globale, senza considerare i diversi stati del gioco:

$$\tilde{a}_i^{(m)} = \frac{1}{3}\,\mathrm{clip}\left(\frac{a_i^{(m)} - \nu_0^{(m)}}{\tau_0^{(m)}},\ 0,\ 3\right) \tag{3.9}$$

La scelta, appena descritta, garantisce che il confronto tra i modelli sia equo, non influenzandolo dalle scelte progettuali di raffinamento relative al modello di Isolation Forest.

3.3.2 Integrazione temporale: accumulo dello stress

Il risultato dell'Equazione 3.8 corrisponde al punteggio $\hat{a}_i$, quest'ultimo indica la misura *istantanea* di una sola finestra da cinque secondi (w_i). Per rendere il movimento della barra più fluido, senza quindi che raggiungesse in continuazione dei picchi isolati, si è pensato ad una deviazione mantenuta nel tempo. Per fare questo introduco la variabile di stato: $\Theta_i \in [0, \Theta_{max}]$. Il suo funzionamento consiste nel suo aggiornamento ad ogni finestra, ma seguendo una dinamica da ***Leaky Integrator***, ovvero un *integratore con perdita costante*, così definito:

$$\Theta_i = \mathrm{clamp}\big(\Theta_{i-1} + \alpha \cdot \hat{a}_i - \beta,\ 0,\ \Theta_{max}\big) \tag{3.10}$$

dove α corrisponde al coefficiente che regola la crescita della barra, β invece è un certo tasso di discesa costante, sottratto costantemente per ogni finestra. Infine, $\mathrm{clamp}(\cdot, 0, \Theta_{max})$ lega il valore all'interno del confine numerico tra zero e il massimo ammissibile. La scelta di utilizzare questa formulazione è stata presa poiché ha un basso costo computazionale e per la sua interpretabilità, proprietà necessarie proprio perché l'aggiornamento doveva essere in tempo reale.

3.3.3 Adattamento del gameplay

Una volta che si raggiunge la soglia massima di stress comportamentale, il valore di Θ diventa l'input della seguente funzione $A : [0, \Theta_{max}] \to \{M_0, M_1\}$, che determina due possibili scenari:

$$A(\Theta_i) = \begin{cases} M_1, & \Theta_i \geq \Theta_{max} \\ M_0, & \text{altrimenti} \end{cases} \tag{3.11}$$

In altre parole se, graficamente, la barra dello stress comportamentale raggiunge la soglia critica, allora il giocatore passa ad un livello di gioco che modifica radicalmente il gameplay, trasformando gli aspetti grafici, sonori e di meccaniche. Nello specifico: avviene un cambio di scena dove il giocatore viene "teletrasportato" in uno spazio di gioco differente, la mappa input-azione vengono invertiti e i parametri fisici si modificano, infatti la gravità si riduce notevolmente.

Al momento della transizione, sia da M_0 a M_1 sia tornando all'indietro, il sistema azzera solamente la variabile di accumulo Θ. La sessione di calibrazione dal lato del server, e insieme ad essa le baseline già stimate (B_0 e le eventuali B_s per-stato), non vengono reinizializzate, anzi restano valide e continuano ad essere utilizzate anche all'interno della modalità "*psichedelica*". Il mondo alterato non costituisce, dal punto di vista del rilevamento, un contesto comportamentale realmente separato da quello ordinario, per quanto lo sia dal punto di vista dell'esperienza videoludica.

Inoltre, è doveroso precisare che il ritorno ad M_0 è un evento di gioco guidato, non dal valore della barra dello stress comportamentale ma da un trigger spaziale. La definizione delle transizioni, definita da un'unica funzione, nello specifico si concretizza in due distinte condizioni:

$$M_0 \longrightarrow M_1 \quad \text{se } \Theta_i \geq \Theta_{max}, \qquad M_1 \longrightarrow M_0 \quad \text{raggiungendo il portale di ritorno} \tag{3.12}$$

3.4 Posizionamento della tesi rispetto allo stato dell'arte

Il metodo descritto, fino ad ora, in questo capitolo si differenzia da altre soluzioni discusse nel Capitolo 2 su tre assi principali. Il primo riguarda la modalità di rilevamento. I sistemi di Dynamic Difficulty Adjustment (DDA) classico [7, 21, 14] regolano la difficoltà videoludica su metriche di performance aggregate, senza costruire nessun tipo di riferimento comportamentale per singolo giocatore, mentre gli approcci di affective computing [10, 3] rilevano lo stato del giocatore in tempo reale; tuttavia, richiedono sensori biometrici dedicati. Il sistema proposto in questa tesi rileva lo stress comportamentale in modo non invasivo e personale, a partire dai soli pattern di input del controller, senza nessuna delle due limitazioni.

Il secondo asse riguarda la personalizzazione della baseline. Nessuno dei sistemi discussi nel Capitolo 2 costruisce un riferimento comportamentale personale, calibrato per singolo giocatore e per singolo stato di gioco, all'interno della stessa sessione di gioco. Il precedente più vicino a questa impostazione proviene da un dominio applicativo estraneo ai videogiochi [15], a conferma che tale granularità di personalizzazione non risulta finora esplorata nel contesto videoludico.

L'ultimo asse riguarda l'ampiezza dell'adattamento. Mentre il DDA classico agisce, di solito, su un numero limitato di parametri legati alla difficoltà, il sistema qui proposto agisce su più livelli contemporaneamente e in modo continuativo a run-time: spazio di gioco, mappa input-azione, parametri fisici e traccia sonora.

In sintesi, gli elementi formalizzati in questo capitolo costituiscono la specifica concettuale del sistema sviluppato. Il Capitolo successivo descrive come ciascuno di questi elementi sia stato realizzato concretamente, rispettivamente nel motore di gioco Godot 4 e nell'infrastruttura sviluppata in Python.

Capitolo 4

Implementazione

4.1 Panoramica dell'architettura

Il sistema sviluppato si organizza in base ad un'architettura di tipo *client-server*. Dalla parte del *client* si ha un'infrastruttura sviluppata in Godot 4, questa si occupa di rilevare l'input del giocatore dal videogioco, inoltre si occupa anche di restituire al giocatore l'adattamento del gameplay qualora la barra dello stress comportamentale raggiunga la soglia massima. Mentre, il server sviluppato in linguaggio Python tramite il framework **FastAPI**, si occupa dell'elaborazione statistica e della stima del punteggio di anomalia comportamentale.

Queste due componenti riescono a comunicare per mezzo di richieste HTTP su una rete e ad una determinata porta in locale. Questa comunicazione avviene tramite richieste alla rete ogni $\Delta t = 5$ secondi. La seguente Figura 4.1 mostra il diagramma a blocchi dell'intera architettura:

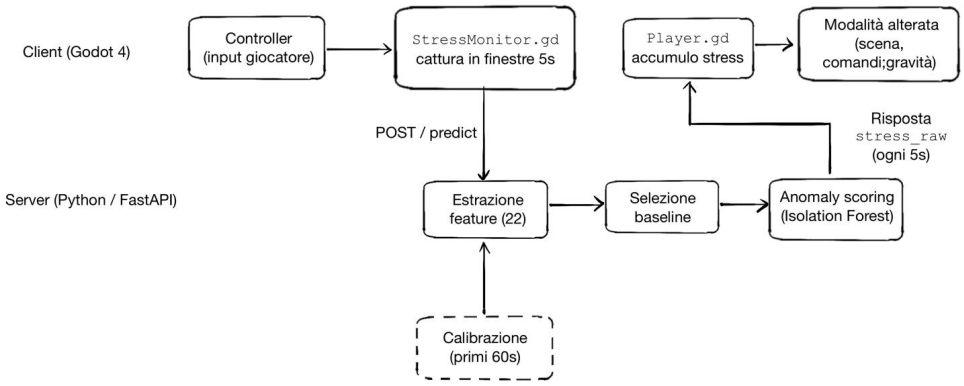


FIGURA 4.1: Architettura generale del sistema: componenti lato client (Godot) e lato server (Python/FastAPI). Il ciclo cattura–richiesta–risposta–adattamento si ripete ogni $\Delta t = 5$ secondi.

Ogni blocco di questo diagramma rappresenta una delle successive sotto sezioni del capitolo:

- Lato **client** troviamo:
  - Blocco "*Controller*": corrisponde all'input del giocatore, nonché sorgente degli eventi di input. Tuttavia, questo blocco non sarà approfondito maggiormente poiché corrisponde ad hardware esterno al lavoro di questo sistema.
  - Blocco "`StressMonitor.gd` cattura finestre di 5s": è descritto nel dettaglio nella Sezione 4.2.1, all'interno della quale si trova anche la Sezione 4.2.2. Entrambe le sezioni si trovano dentro lo stesso script.
- Lato **server** troviamo:
  - Blocco "*Estrazione feature 22*": che viene descritto nella Sezione 4.3.1.
  - Blocco "*Calibrazione (primi 60s)*": questa parte del diagramma ha i bordi tratteggiati, ad indicare che la sua vita dura solamente i primi 60s della sessione relativamente alla calibrazione iniziale.
  - Blocco "*Selezione baseline*": come per il punto precedente, entrambi i blocchi rientrano nella Sezione 4.3.2.

  - Blocchi "*Anomaly scoring (Isolation Forest)*" e "`Player.gd` accumulo stress": rientrano entrambi nella Sezione 4.4.1, rispettivamente trattano del calcolo del punteggio finale e la sua integrazione temporale all'interno della sessione.
  - Blocco "*Modalità alterata (scena, comandi, gravità)*": descritto nella Sezione 4.5.1, che descrive come è stata implementata la meccanica adattiva all'interno del gameplay.

- Sezione 4.6.1, che in modo più generale descrive tutto il ciclo end-to-end rappresentato, nel diagramma, dalle frecce direzionate.
- Sezione 4.6.2: all'interno della quale si trova l'elenco delle diverse librerie utilizzate per lo sviluppo del sistema.

4.2 Acquisizione e definizione dell'input

4.2.1 Cattura e trasmissione dell'input

L'acquisizione dell'input all'interno del progetto è completamente gestita dallo script `StressMonitor.gd`, salvato come un *autoload* di Godot (ovvero, corrisponde ad un nodo di tipo globale che viene istanziato ad ogni avvio del gioco in modo totalmente autonomo, inoltre il suo contenuto è accessibile in qualsiasi punto, qualora si volesse accedervi). La scelta di questa implementazione è particolarmente trasversale al compito di questo script, infatti `StressMonitor.gd` deve poter comunicare con `Player.gd` senza nessun tipo di dipendenza tra i due nodi (StressMonitor e Player).

La Figura 4.2 mostra la configurazione di `StressMonitor` come autoload nel pannello `Project Settings` in Godot:

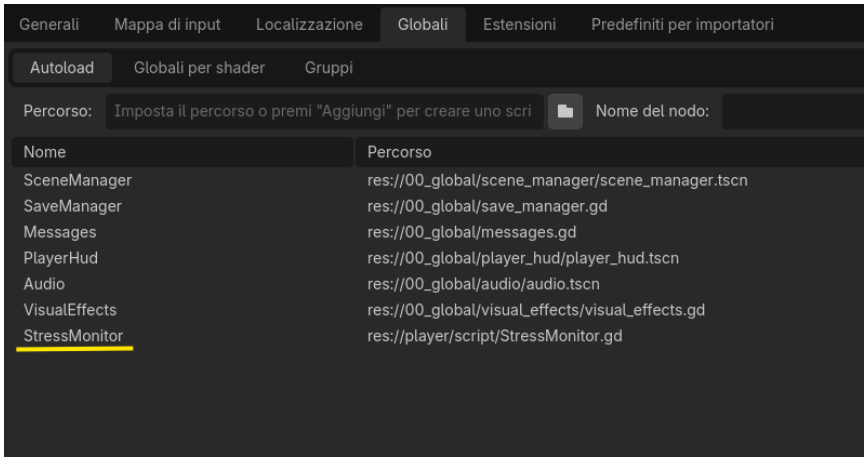


FIGURA 4.2: Configurazione di `StressMonitor.gd` come autoload nel pannello Project Settings.

L'input dei vari eventi accade per mezzo della callback `_input()`, che viene richiamata dall'engine ogni volta che si genera un evento durante l'esecuzione del frame corrente. L'input può essere catturato da tre diverse sorgenti: pulsanti di un controller, stick analogici e tastiera (quest'ultima come fallback, in modo che anche senza collegare un controller il giocatore possa sempre utilizzare la tastiera).

Listing 1: Cattura degli eventi di input in `StressMonitor.gd`

```
func _input(event: InputEvent) -> void:
    var t := (Time.get_ticks_msec() / 1000.0) - _window_start_time

    if event is InputEventJoypadButton:
        _input_used_this_window = true
        if event.button_index in TRACKED_BUTTONS:
            _buffered_events.append({
                "type": "key",
                "event": str(event.button_index),
                "state": 1.0 if event.pressed else 0.0,
                "t": t,
            })

    elif event is InputEventKey and not event.echo:
        if event.keycode in TRACKED_KEYS:
            _input_used_this_window = true
            _buffered_events.append({
                "type": "key",
                "event": "kb_" + str(event.keycode),
                "state": 1.0 if event.pressed else 0.0,
                "t": t,
            })

    elif event is InputEventJoypadMotion:
        if event.axis in TRACKED_AXES:
            var prev = _last_axis_value.get(event.axis, 0.0)
            if abs(event.axis_value - prev) > 0.02:
                _input_used_this_window = true
                _buffered_events.append({
                    "type": "absolute",
                    "event": str(event.axis),
                    "state": event.axis_value,
                    "t": t,
                })
                _last_axis_value[event.axis] = event.axis_value
```

Come si nota nell'ultima condizione, è stata applicata una soglia di 0.02 sulla variazione dello stick analogico. Infatti, senza questa condizione lo stick conserverebbe un rumore residuo, il quale potrebbe generare degli eventi accidentali che sarebbero senza significato comportamentale, ma solo un errore dato dall'hardware, modificando le feature calcolate dal sistema.

Ognuno degli eventi viene registrato e allocato in un buffer in locale, ma quest'ultimo non viene trasmesso subito; anzi, la trasmissione segue intervalli regolari ($\Delta t = 5$ secondi `WINDOW_SECONDS`), che corrisponde alla costante citata anche nella Sezione 3.2.1, nel seguente ciclo `_process()`:

LISTING 2: Invio periodico della finestra al server

```
func _process(_delta: float) -> void:
    var elapsed := (Time.get_ticks_msec() / 1000.0) -
    _window_start_time
    if elapsed >= WINDOW_SECONDS:
        _send_window()


func _send_window() -> void:
    var events_copy := _buffered_events.duplicate(true)
    _buffered_events.clear()
    _input_used_this_window = false
    _window_start_time = Time.get_ticks_msec() / 1000.0

    if events_copy.is_empty():
        return

    var body := JSON.stringify({"events": events_copy, "game_state":
    current_game_state})
    var headers := ["Content-Type: application/json"]
    _http.request(SERVER_URL, headers, HTTPClient.METHOD_POST, body)
```

Si noti come lavora la condizione d'uscita anticipata se `events_copy` è vuoto. Infatti, quando il giocatore non ha prodotto nessun tipo di input in una finestra, non viene effettuata nessuna richiesta al server, in modo da non sprecare chiamate di rete poiché verrebbero introdotti campioni senza alcun significato comportamentale.

Per concretizzare come sono formati i dati scambiati, di seguito si mostra un esempio `illustrativo` del corpo di una richiesta `POST /predict` nel Listato 3, costruito appositamente per questo elaborato. Quest'ultima è relativa ad una finestra durante uno scontro con un avversario che appartiene alla categoria dei "*boss*", lo scambio contiene due pressioni ravvicinate dello stesso pulsante, un movimento ampio dello stick analogico del controller in entrambe le direzioni e la pressione di un secondo pulsante.

LISTING 3: Esempio di corpo della richiesta inviata a `/predict`

```
{
    "events": [
    {"type": "key", "event": "0", "state": 1.0, "t": 0.12},
    {"type": "key", "event": "0", "state": 0.0, "t": 0.19},
    {"type": "key", "event": "0", "state": 1.0, "t": 0.41},
    {"type": "key", "event": "0", "state": 0.0, "t": 0.47},
    {"type": "absolute", "event": "0", "state": 0.86, "t": 0
.55},
    {"type": "absolute", "event": "0", "state": -0.92, "t": 1
.20},
    {"type": "key", "event": "2", "state": 1.0, "t": 2.03},
    {"type": "key", "event": "2", "state": 0.0, "t": 2.10}
    ],
    "game_state": "combat_boss"
}
```

Il campo "*event*" riporta l'indice numerico del pulsante oppure dell'asse coinvolto, mentre `state` ha il valore di 1.0/0.0 per la pressione o il rilascio di un tasto, o il valore continuo dell'asse per un movimento dello stick.

Parallelamente all'invio di finestre, ciò che corrisponde al flusso principale del sistema, `StressMonitor.gd` contiene, anche, una funzione di log generica, che ha il compito di registrare degli eventi, ma indipendentemente dal ciclo di finestre da 5 secondi:

LISTING 4: Funzione di logging eventi in `StressMonitor.gd`

```
1  func log_game_event(event_name: String, value: float = 0.0) ->
   void:
2      var http := HTTPRequest.new()
3      add_child(http)
4      http.request_completed.connect(func(_r, _c, _h, _b):
   http.queue_free())
5      var body := JSON.stringify({"event_name": event_name, "value":
   value})
6      http.request(LOG_EVENT_URL, ["Content-Type:
   application/json"], HTTPClient.METHOD_POST, body)
```

Come si può notare, la funzione prende in input il nome di un qualsiasi evento e annesso un valore numerico. Quindi vengono inviati al server per mezzo di una richiesta HTTP, ma in modo asincrono rispetto al normale log delle finestre. Questa funzione, allo stato attuale del sistema, viene chiamata solamente per marcare l'inizio della fase di calma e l'inizio di quella frenetica (`calm_start`, `frenetic_start`), dinamica che sarà spiegata approfonditamente nel Capitolo 5. L'esistenza di questa funzione non ha come unico scopo quello appena descritto; infatti, la stessa potrebbe essere richiamata anche in corrispondenza di altri eventi di gameplay, come la morte del giocatore o il danno subito da quest'ultimo, tutto allo scopo di arricchire il grafico dello stress nel tempo, oltre alla presenza dei marker di calma/frenesia.

4.2.2 Classificazione dello stato di gioco

Nella Sezione 3.2.1 è stato formalizzato come ognuna delle finestre da 5s w_i viene associata ad uno stato di gioco $\sigma(w_i) \in S$. Questo meccanismo viene gestito, nuovamente, da `StressMonitor.gd`, tramite una dinamica a *due livelli*:

- Una mappatura più statica che parte dallo stato "grezzo" del personaggio a categoria generale,
- Una distinzione specifica per quanto riguarda il combattimento.

LISTING 5: Mappatura degli stati grezzi in `StressMonitor.gd`

```
const STATE_GROUPS := {
    "attack": "combat",
    "take_damage": "combat",
    "death": "combat",
    "jump": "traversal",
    "fall": "traversal",
    "dash": "traversal",
    "run": "traversal",
    "crouch": "traversal",
    "idle": "idle",
}


func set_game_state(raw_state_name: String) -> void:
    var key := raw_state_name.to_lower()
    var category: String = STATE_GROUPS.get(key, "unknown")

    if category == "combat":
        var boss_present := not
get_tree().get_nodes_in_group("Boss").is_empty()
        category = "combat_boss" if boss_present else
"combat_regular"

    current_game_state = category
```

La funzione, sopra riportata, `set_game_state()` viene chiamata da `Player.gd` per ogni transizione della *state machine* del personaggio di gioco, la funzione riceve in input il nome dello stato in cui si trova il personaggio (per esempio `"Attack"`, `"Idle"`).

Subito dopo, il nome viene riscritto in minuscolo tramite `to_lower()`, in modo da renderlo indipendente dalla convenzione usata da Godot, quindi per permettere così di cercare il nome all'interno di `STATE_GROUPS`. Di seguito la Tabella 4.1 mostra la mappatura completa degli stati:

Stato `Player.gd`	**Categoria intermedia**	**Categoria finale** $\in S$

`Attack`
`TakeDamage`
`Death`
	combat	`combat_regular` / `combat_boss`

`Jump`
`Fall`
`Dash`
`Run`
`Crouch`
	traversal	traversal

`Idle`	idle	idle
(qualsiasi altro nome)	–	`unknown` $\rightarrow$ fallback su B_0

TABELLA 4.1: Mappatura dagli stati grezzi di `Player.gd` alle categorie di S.

Come si può notare, l'ultima riga evidenzia un caso precedentemente discusso (Sezione 3.2.1), ovvero: quando il nome dello stato NON compare tra quelli già presenti in `STATE_GROUPS`, la funzione `.get()` restituisce in output un valore `"unknown"`. Il codice non tratta in modo speciale questo valore, infatti: viene considerato e trattato come un qualunque altro stato che non è stato ancora calibrato, che ricade sulla baseline globale B_0 solo perché, empiricamente, non raggiunge mai la soglia minima k_{min} all'interno di una sessione.

Per quanto riguarda lo stato di combattimento, la distinzione (motivata meglio nella Sezione 3.2.2) tra la categoria `combat_boss` e quella `combat_regular` è stata effettuata interrogando i gruppi Godot dell'*albero delle scene* grazie alla funzione `get_nodes_in_group("Boss")` che una volta rilevato un nodo combattimento designa in quale categoria specifica ricade. Per la realizzazione di questa dinamica è stato necessario trovare un modo per aggiungere esplicitamente determinati NPC nemici al gruppo `"Boss"` dal pannello dell'engine Godot, se non ci fosse stata un'opzione del genere ci sarebbe stata una classificazione errata dei combattimenti.

La Figura 4.3 mostra l'assegnazione di un avversario del tipo "*boss*" al gruppo Godot `"Boss"` tramite il pannello `Node` → `Groups`:

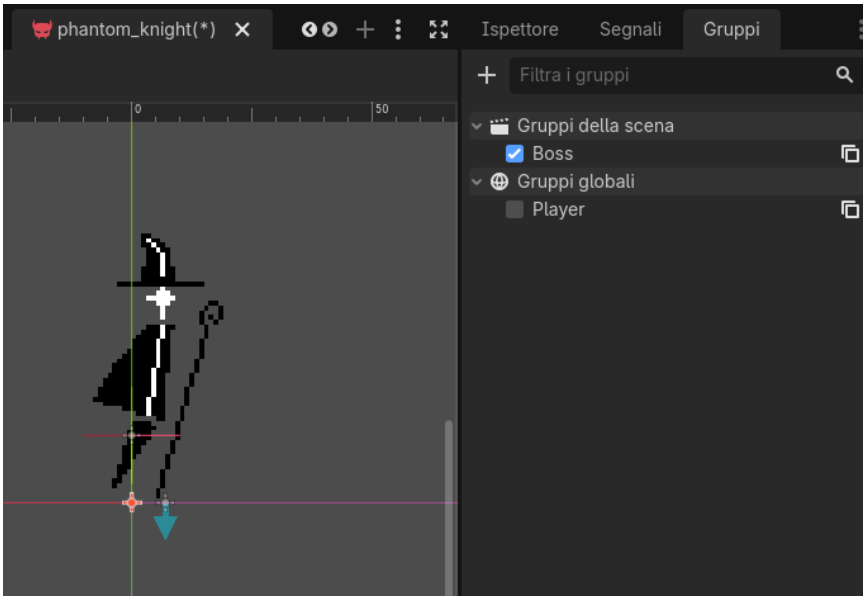


FIGURA 4.3: Assegnazione di un nemico boss al gruppo Godot `"Boss"`.

4.3 Elaborazione statistica e calibrazione lato server

4.3.1 Estrazione delle feature

Il prelevamento del vettore di feature $x_i \in \mathbb{R}^{22}$, descritto formalmente nella Sezione 3.2.1 tramite la funzione φ, è implementato dalla funzione `compute_features` lato server in Python. La funzione riceve in input la lista di eventi grezzi all'interno di un buffer dal client (il gioco) per la finestra corrente, e li separa, innanzitutto, in base alla tipologia:

LISTING 6: Separazione degli eventi e statistiche sui tasti in `compute_features()`

```
def compute_features(events: List[ControllerEvent]) -> dict:
    keys = [e for e in events if e.type == "key"]
    abss = [e for e in events if e.type == "absolute"]

    key_press_count = sum(1 for e in keys if e.state == 1)
    key_release_count = sum(1 for e in keys if e.state == 0)
    unique_keys = len(set(e.event for e in keys))
    key_frequency = key_press_count / WINDOW_SECONDS

    press_times = sorted(e.t for e in keys if e.state == 1)
    if len(press_times) >= 2:
        intervals = np.diff(press_times)
        avg_inter_press = float(np.mean(intervals))
        std_inter_press = float(np.std(intervals)) if len(intervals) >
1 else 0.0
    else:
        avg_inter_press = 0.0
        std_inter_press = 0.0

    burstiness = (std_inter_press / (avg_inter_press + 1e-6)) if
avg_inter_press > 0 else 0.0
```

Le diverse feature collegate ai tasti derivano dalla lista `keys`: il numero di pressioni e rilasci corrisponde ad un filtro sul campo `state`, mentre `avg_inter_press` e `std_inter_press` richiedono di ordinare cronologicamente i momenti di pressione e calcolare gli intervalli consecutivi tramite `np.diff()`.

La `burstiness` corrisponde al rapporto tra la deviazione standard e la media di questi intervalli, con un piccolo termine (10^{-6}) al denominatore per evitare la divisione per zero quando la media è nulla: quindi valori molto grandi indicano intervalli molto irregolari (pressioni concentrate in raffiche), valori prossimi a zero con un ritmo di pressione regolare.

Le feature legate allo stick analogico del controller seguono una logica uguale sulla lista `abss`:

LISTING 7: Statistiche sul movimento dello stick in `compute_features()`

```
if abss:
    vals = np.array([e.state for e in abss])
    stick_mean = float(vals.mean())
    stick_std = float(vals.std()) if len(vals) > 1 else 0.0
    stick_max = float(np.abs(vals).max())
    stick_count = len(vals)
    stick_range = float(vals.max() - vals.min())
    movement_energy = float(np.sum(vals ** 2))
    sign_changes = int(np.sum(np.diff(np.sign(vals)) != 0)) if
len(vals) > 2 else 0
else:
    stick_mean = stick_std = stick_max = stick_range = 0.0
    stick_count = 0
    movement_energy = 0.0
    sign_changes = 0

action_intensity = key_press_count + stick_count
key_diversity_rate = unique_keys / (key_press_count + 1)
```

É importante indicare la definizione di `movement_energy` come la somma dei quadrati dei valori rilevati dallo stick del controller (energia in senso fisico, non semplice ampiezza media), e di `sign_changes` come il numero di inversioni di segno consecutive nella sequenza dei valori, un proxy diretto di quanto il movimento sia oscillatorio invece che direzionale. Il termine `+1` al denominatore di `key_diversity_rate` evita la stessa divisione per zero, ma questa volta per il caso di nessuna pressione all'interno di una finestra.

La casualità dei tasti premuti (`key_entropy`) viene calcolata a partire dalla distribuzione della frequenza relativa di ogni distinto tasto nella finestra, secondo la definizione standard di entropia di Shannon in base 2, anch'essa con un termine di stabilità numerica (10^{-9}) per evitare il logaritmo di zero.

Un sottoinsieme delle 22 feature, le cinque differenziali elencate nella tabella 3.1, richiede una memoria delle finestre precedenti, che non è disponibile calcolando la sola finestra corrente. Quindi questa, appena citata, memoria viene mantenuta all'interno di una struttura di stato globale, condivisa tra le chiamate successive della funzione:

LISTING 8: Feature differenziali e a media mobile

```
delta_key_press = key_press_count - state["prev_key_press"]
delta_key_freq = key_frequency - state["prev_key_freq"]
delta_stick_std = stick_std - state["prev_stick_std"]

state["key_freq_history"].append(key_frequency)
state["stick_std_history"].append(stick_std)
rolling_key_freq = float(np.mean(state["key_freq_history"]))
rolling_stick_variability = float(np.mean(state["stick_std_history"]))

state["prev_key_press"] = key_press_count
state["prev_key_freq"] = key_frequency
state["prev_stick_std"] = stick_std
```

Dove le tre feature che iniziano per `delta_*` confrontano il valore corrente con il valore memorizzato nella chiamata precedente, mentre le feature che iniziano per `rolling_*` calcolano una media su una coda che ha lunghezza fissa (pari alle ultime tre finestre, quindi 15 secondi). Questa dipendenza implica che la funzione `compute_feature()` non è una funzione "pura": ovvero due delle chiamate che hanno lo stesso input possono restituire risultati differenti in base alla cronologia delle chiamate precedenti in una stessa sessione, una caratteristica logica con la natura stessa delle feature differenziali, che per definizione dipendono proprio dal passato recente della sessione.

Il significato di ogni feature tra le 22, così come la loro "categorizzazione concettuale", è stato già descritto nella tabella del Capitolo precedente (3); in questa sezione si è descritta solamente la realizzazione computazionale.

4.3.2 Calibrazione e individuazione della baseline

Questa sottosezione descrive come si compone l'implementazione dei meccanismi di calibrazione descritti nel dettaglio nella Sezione 3.2.2: la baseline globale B_0, stimata partendo dalla fase iniziale della sessione di gioco a prescindere dallo stato di gioco, le baseline "per-stato" B_s, vengono poi stimate separatamente per ogni categoria $s \in S$ nel momento in cui viene raggiunta la soglia k_{min}.

Calibrazione globale

La calibrazione iniziale globale viene eseguita dalla funzione `run_calibration()`, quest'ultima viene chiamata una sola volta alla fine delle prime `CALIBRATION_WINDOWS = 12` finestre della sessione (equivalenti ai primi 60 secondi, con $\Delta t = 5$ secondi):

LISTING 9: Calibrazione della baseline globale B_0

```
def run_calibration():
    buf = state["calibration_buffer"]
    X = np.array([[row[f] for f in STRESS_FEATURES] for row in buf])

    mean = X.mean(axis=0)
    std = X.std(axis=0)
    std[std == 0] = 1e-6  # evita divisioni per zero su feature
costanti in calibrazione
    state["baseline_mean"] = mean
    state["baseline_std"] = std

    candidates = build_candidate_models()
    for name, model in candidates.items():
        feat_list = MODEL_FEATURE_SETS[name]
        X_model = np.array([[row[f] for f in feat_list] for row in buf])
        model.fit(X_model)
        calib_scores = -model.score_samples(X_model) if hasattr(model,
"score_samples") else -model.decision_function(X_model)
        abn_mean = float(calib_scores.mean())
        abn_std = float(calib_scores.std()) if calib_scores.std() > 0
else 1.0
        state["models"][name] = {"model": model, "abn_mean": abn_mean,
"abn_std": abn_std}

    state["calibrated"] = True
```

Questa funzione svolge due differenti compiti sugli stessi dati di calibrazione: viene calcolata la media e la deviazione standard per ognuna delle 22 feature (`baseline_mean`, `baseline_std`), che corrispondono a μ_0 e σ_0 (nella notazione del Capitolo precedente 3), inoltre, per ognuno dei quattro modelli di *anomaly detection* dichiarati in `build_candidate_models()` (Sezione 4.4.1), la funzione li addestra sui dati grezzi di calibrazione e ne memorizza separatamente media e deviazione standard dei punteggi di anomalia prodotti (`abn_mean`, `abn_std`): sono questi ultimi due valori a costituire ν_0 e τ_0, utilizzati nella normalizzazione a 3σ del raffronto tra i diversi modelli (quest'ultima parte viene poi discussa nel Capitolo 5).

É doveroso notare come ognuno dei modelli viene addestrato sul **proprio sottoinsieme di feature** (`MODEL_FEATURE_SETS`): su quattro modelli, tre usano le 22 feature complete, mentre l'algoritmo *Elliptic Envelope* utilizza un sottoinsieme ristretto di solamente 5 feature, per la ragione di instabilità numerica già discussa nel Capitolo precedente.

Calibrazione per stato

Le diverse baseline B_s sono gestite dalla funzione `update_state_buffer_and_maybe_fit()`, chiamata per ogni finestra con lo stato di gioco corrente:

LISTING 10: Calibrazione e congelamento delle baseline per-stato B_s

```
def update_state_buffer_and_maybe_fit(game_state: str, feats: dict):
    if game_state in state["state_models"]:
        return  # baseline di questo stato gia' calibrata: resta fissa
per il resto della sessione

    buf = state["state_buffers"][game_state]
    buf.append(feats)

    if len(buf) < STATE_MIN_SAMPLES:
        return  # non ancora abbastanza dati per questo stato specifico

    X = np.array([[row[f] for f in STRESS_FEATURES] for row in buf])
    mean = X.mean(axis=0)
    std = X.std(axis=0)
    std[std == 0] = 1e-6

    model = IsolationForest(n_estimators=100, contamination="auto",
random_state=42)
    model.fit(X)
    calib_scores = -model.score_samples(X)
    abn_mean = float(calib_scores.mean())
    abn_std = float(calib_scores.std()) if calib_scores.std() > 0 else
1.0

    state["state_models"][game_state] = {
        "model": model, "mean": mean, "std": std,
        "abn_mean": abn_mean, "abn_std": abn_std, "n_samples": len(buf),
    }
```

Il controllo effettuato nella prima condizione (`if game_state in state["state_models"]: return`) corrisponde all'implementazione diretta della condizione dell'Equazione 3.4: se una baseline è stata già formata, la funzione termina ed esce immediatamente senza ricalcolare niente, creando il congelamento discusso nella Sezione 3.2.2.

Solamente quando il buffer di uno stato specifico raggiunge k_{min} = `STATE_MIN_SAMPLES = 8` campioni, viene addestrato un modello di *Isolation Forest* dedicato solamente a quello stato, a differenza della calibrazione globale, dove vengono addestrati tutti e quattro i modelli di confronto. Questa differenza è data dal fatto che le baseline per-stato servono solamente a pilotare il gameplay reale, e non entrano nel confronto sperimentale del Capitolo 5.

Una differenza rispetto alla calibrazione globale: in questo punto il modello di *Isolation Forest* è istanziato con `n_estimators=100`, contro i `150` utilizzati in `build_candidate_models()` per la calibrazione globale, quest'ultima è una scelta implementativa minore, atta a contenere il costo computazionale della ricalibrazione per ogni stato, quest'ultima può avvenire più volte nel corso di una sessione (una volta per ogni stato che arriva alla soglia), a differenza della calibrazione globale che avviene una singola volta per sessione.

4.4 Punteggio dell'anomalia e accumulazione dello stress

4.4.1 Calcolo del punteggio dell'anomalia

Il calcolo della magnitudo e della direzione, che sono stati formalizzati nelle Equazioni 3.6 e 3.7 del Capitolo 3, è poi implementato dalla funzione `score_with_bundle()`, la quale riceve la baseline in input da usare (sia essa globale o per-stato) e il vettore delle feature dalla finestra corrente:

Listing 11: Calcolo di magnitudine, direzione e punteggio finale

```
def score_with_bundle(bundle: dict, feats: dict):
    x_vec = np.array([feats[f] for f in STRESS_FEATURES])
    x = x_vec.reshape(1, -1)

    abnormality = raw_anomaly_score(bundle["model"], x)
    normalized = (abnormality - bundle["abn_mean"]) / bundle["abn_std"]

    magnitude = float(np.clip(normalized, 0.0, 2.0) / 2.0)

    z = (x_vec - bundle["mean"]) / bundle["std"]
```

```
    composite_direction = float(np.mean(z))
    top_idx = int(np.argmax(np.abs(z)))
    top_feature = STRESS_FEATURES[top_idx]
    top_zscore = float(z[top_idx])


    score = magnitude if composite_direction > 0 else magnitude * 0.3
    return score, top_feature, top_zscore
```

Il rapporto con la formalizzazione nel Capitolo 3 è diretto: `magnitude` implementa $\hat{m}_i$ (Equazione 3.6), poi `composite_direction` implementa d_i (Equazione 3.7), e infine l'ultima riga della funzione implementa il fattore di asimmetria, ovvero: $\lambda = 0.3$ dell'Equazione 3.8. La variabile `top_feature` inquadra la feature con lo z-score più grande, in valore assoluto, utilizzata a scopo di spiegabilità nel Capitolo 5.

La selezione tra: baseline globale e per-stato (ovvero Equazione 3.4) viene effettuata dalla funzione `score_official()` qui riportata:

LISTING 12: Selezione della baseline attiva per il punteggio ufficiale

```
def score_official(game_state: str, feats: dict):
    state_bundle = state["state_models"].get(game_state)
    if state_bundle is not None:
        score, top_feature, top_zscore =
score_with_bundle(state_bundle, feats)
        baseline_used =
f"per_stato:{game_state}(n={state_bundle['n_samples']})"
        return score, top_feature, top_zscore, baseline_used

    global_bundle = {
        "model": state["models"][OFFICIAL_MODEL_NAME]["model"],
        "mean": state["baseline_mean"],
        "std": state["baseline_std"],
        "abn_mean": state["models"][OFFICIAL_MODEL_NAME]["abn_mean"],
        "abn_std": state["models"][OFFICIAL_MODEL_NAME]["abn_std"],
    }
    score, top_feature, top_zscore = score_with_bundle(global_bundle,
feats)
    return score, top_feature, top_zscore, "globale(riserva)"
```

Il valore `baseline_used` restituito in output da questa funzione viene poi salvato nel log della sessione ad ogni finestra, rendendo possibile, nella fase di analisi nel Capitolo 5, verificare successivamente quali finestre all'interno di una sessione sono state valutate con baseline per-stato e quali con quella globale di riserva.

Di fianco al punteggio ufficiale, il sistema calcola parallelamente, su una stessa finestra, il punteggio dei quattro diversi modelli di confronto tramite la funzione `score_all_models()`, in base alla normalizzazione a 3σ descritta formalmente nel Capitolo 3, una discussione già affrontata nel capitolo prima citato, qui semplicemente viene confermata dal punto di vista del codice Python: il punteggio ufficiale usa perennemente `score_with_bundle()` con un livello di saturazione pari a 2σ ed eventuale baseline per-stato, mentre il confronto usa sempre la baseline globale con un livello di saturazione a 3σ, prescindendo lo stato di gioco.

Il seguente Listato 13, di natura illustrativa come il Listato precedente, evidenzia un esempio di una risposta completa riconsegnata da `/predict` per la finestra dell'esempio precedente (ovvero Listato 3, nella Sezione 4.2.1), dove la baseline per lo stato `combat_boss` risulta già considerata. I valori numerici non sono reali, ma coerenti con le diverse formule già descritte nel Capitolo 3.

LISTING 13: Esempio di risposta restituita da `/predict`

```
{
    "phase": "active",
    "stress_raw": 3.42,
    "stress_score_debug": 0.605,
    "top_contributing_feature": "burstiness",
    "top_feature_zscore": 2.184,
    "comparison_scores": {
        "isolation_forest": 0.58,
        "one_class_svm": 0.51,
        "local_outlier_factor": 0.63,
        "mahalanobis": 0.47
    },
    "baseline_used": "per_stato:combat_boss(n=14)",
    "game_state": "combat_boss"
}
```

Da questo esempio, il valore di `stress_raw` è quello realmente utilizzato nello script di `Player.gd` per l'accumulo (Sezione 4.4.2); si noti come, mettendo in pratica la relazione $\text{stress_raw} = 1 + 4 \cdot \text{official_score}$, il valore 3.42 corrisponde a `stress_score_debug` $= 0.605$.

All'interno di questo listato il campo `top_contributing_feature` mostra come, nella finestra, la deviazione più rilevante in confronto alla baseline riguarda la `burstiness`, che è coerente con l'esempio di input mostrato nel Listato precedente, il quale era maggiormente dominato da pressioni molto ravvicinate dello stesso pulsante. Il campo `comparison_scores` riporta i 4 punteggi calcolati con la normalizzazione a 3σ, argomento già affrontato in precedenza.

4.4.2 Accumulo dello stress nel tempo

Appena dopo l'output, lato server, il punteggio istantaneo $\hat{a}_i$ va in input al client rimappato su un intervallo $[1, 5]$ come `stress_raw`. La meccanica di accumulo dell'Equazione 3.10 del Capitolo 3 è implementata in `Player.gd`:

Listing 14: Accumulo temporale dello stress in `Player.gd`

```
1  const STRESS_INCREMENT := 3.0
2  const STRESS_DECAY := 0.3
3
4  func _on_stress_multiplier_updated(_mult: float, raw_value: float)
   -> void:
5      var score := (raw_value - 1.0) / 4.0
6      stress += STRESS_INCREMENT * score - STRESS_DECAY
```

Come si può osservare la prima riga della funzione ribalta la rimappatura $[1, 5] \rightarrow [0, 1]$ applicata dal server, recuperando però il punteggio $\hat{a}_i \in [0, 1]$; la seconda riga della funzione implementa in modo diretto l'Equazione 3.10, con $\alpha =$ `STRESS_INCREMENT` $= 3.0$ e $\beta =$ `STRESS_DECAY` $= 0.3$. Il vincolo di clamp nell'intervallo $[0, \Theta_{max}]$ e la verifica del superamento della soglia, tutti e due previsti dall'Equazione 3.10 e dalla Sezione 3.3.3, sono gestiti da un `setter` dedicato dalla variabile `stress` descritto nella Sezione 4.5.1.

Le due Figure 4.4 e 4.5 mostrano la barra dello stress comportamentale in due diversi momenti all'interno di una sessione di gioco: ad un livello basso durante una fase di gioco più calma, e vicina alla soglia critica poco prima della transizione:

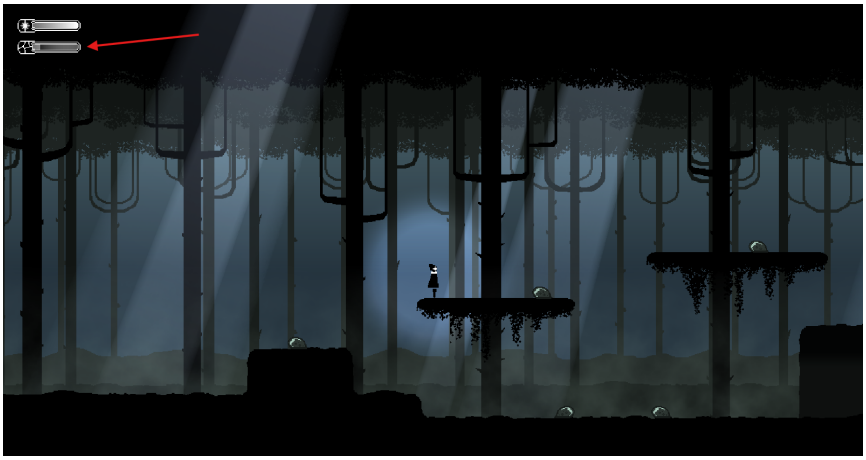


FIGURA 4.4: La barra dello stress comportamentale in un momento in cui è bassa all'interno di una sessione.

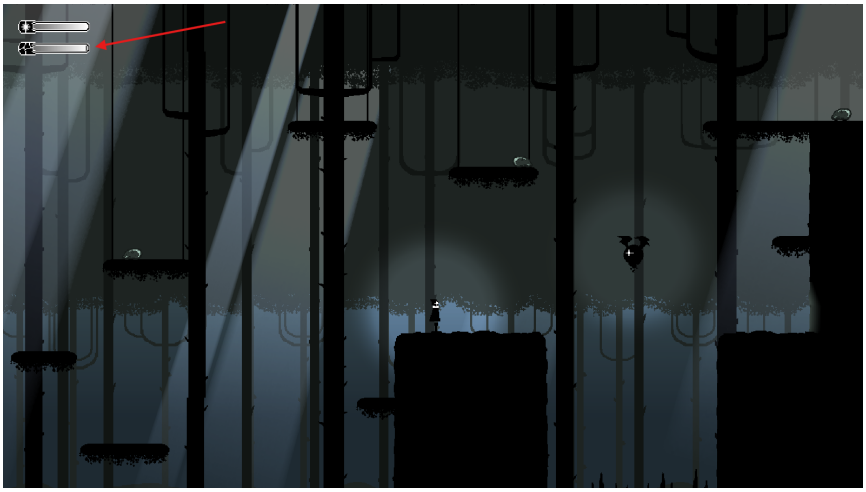


FIGURA 4.5: La barra dello stress comportamentale in un momento in cui è alta, quasi al limite all'interno di una sessione.

4.5 Adattività del gameplay

4.5.1 Mondo psichedelico

Questa sottosezione descrive l'implementazione della funzione di adattività con soglia Θ_i (Sezione 3.3.3) del Capitolo 3, dal quale Godot prende il nome gergale di "*mondo psichedelico*".

Il trigger

Il superamento della soglia critica Θ_{max} è verificato in modo diretto nel `setter` della variabile `stress` di `Player.gd`, ad ogni suo aggiornamento (Sezione 4.4.2):

LISTING 15: Verifica della soglia nel setter di `stress`

```
    var stress : float = 20 :
        set(value):
            stress = clampf(value, 0, max_stress)
            Messages.player_stress_changed.emit(stress, max_stress)
            if stress >= max_stress and not _stress_teleport_triggered:
                _stress_teleport_triggered = true
                _teleport_due_to_stress()

```

Il vincolo `clampf(value, 0, max_stress)` compie il clamp$(\cdot, 0, \Theta_{max})$ dell'Equazione 3.10; la successiva condizione implementa la funzione a soglia $A(\Theta_i)$, con la variabile di tipo booleano `_stress_teleport_triggered` che previene richiami multipli della transizione mentre `stress` resta al massimo.

Trasformazione dell'ambiente di gioco

Uno dei tre livelli che subisce una trasformazione, corrisponde al cambio della scena, gestito da `_teleport_due_to_stress()`:

LISTING 16: Cambio di scena al superamento della soglia

```
1   func _teleport_due_to_stress() -> void:
2       _pre_stress_scene_path =
    get_tree().current_scene.scene_file_path
3       _pre_stress_position = global_position
4
5       get_tree().change_scene_to_file( stress_teleport_scene )
6       await get_tree().scene_changed
7       await get_tree().process_frame
8
9       var spawn_point : Node = get_tree().current_scene.find_child(
    "StressSpawnPoint", true, false )
10      if spawn_point:
11          global_position = spawn_point.global_position
12      else:
13          push_warning( "StressSpawnPoint non trovato in " +
    stress_teleport_scene )
14
15      SceneManager.current_scene_uid = ResourceUID.path_to_uid(
    stress_teleport_scene )
16      SceneManager.scene_entered.emit(
    SceneManager.current_scene_uid )
17      stress = 0
18      _stress_teleport_triggered = false
19
20      SceneManager.load_scene_finished.emit()
21      _apply_psychedelic_effects( true )
22      pass
```

Prima che la scena venga cambiata, la funzione salva scena corrente e la medesima posizione (`_pre_stress_scene_path`, `_pre_stress_position`), in modo tale da ripristinare lo stesso esatto punto al ritorno. Il punto di spawn nella nuova scena "*psichedelica*" viene individuato cercando un nodo che prende il nome di `"StressSpawnPoint"`, convenzione che richiede che ogni scena utilizzabile come "mondo alterato" ne contenga uno; se il nodo non viene trovato, la funzione emette un avviso (`push_warning`) anziché fallire in silenzio.

Come trattato in precedenza nella Sezione 3.3.3 nel Capitolo 3, la trasformazione dell'ambiente azzera solamente la variabile di accumulo `stress`: la sessione di calibrazione lato server, e insieme le diverse baseline già stimate, non vengono reinizializzate ma restano attive anche all'interno del mondo "*psichedelico*". La gestione della scena (`SceneManager.current_scene_uid`, `SceneManager.scene_entered.emit(...)`) è affidata a un sistema di supporto già presente nel progetto, il quale notifica il cambio di scena al resto del gioco; non fa parte del sistema di rilevamento comportamentale e non viene approfondito ulteriormente in questa sede. Il percorso di ritorno verso il mondo "normale" è gestito dalla funzione simmetrica `return_from_stress()`, non riportata poiché conserva la stessa logica di azzeramento del solo `stress`.

Trasformazioni di mappa input-azione e fisica

Oltre al livello appena descritto gli altri due, legati alla trasformazione, si attivano dalla funzione `_apply_psychedelic_effects()`:

LISTING 17: Attivazione combinata di inversione input e gravità alterata

```
1    func _apply_psychedelic_effects( active : bool ) -> void:
2        StressMonitor.set_input_inversion( active )
3        psychedelic_gravity_scale = 0.55 if active else 1.0
4        pass
```

L'inversione dei comandi di input è assegnata allo script `StressMonitor.gd`, che inverte gli eventi associati alle azioni di input `"left"` e `"right"` direttamente nella `InputMap` di Godot:

LISTING 18: Scambio degli eventi di input in `StressMonitor.gd`

```
    func set_input_inversion( active : bool ) -> void:
        if active == inputs_inverted:
            return
        _swap_input_actions( "left", "right" )
        inputs_inverted = active
        pass

    func _swap_input_actions( action_a : String, action_b : String )
    -> void:
        var events_a := InputMap.action_get_events( action_a )
        var events_b := InputMap.action_get_events( action_b )
        InputMap.action_erase_events( action_a )
        InputMap.action_erase_events( action_b )
        for e in events_b:
            InputMap.action_add_event( action_a, e )
        for e in events_a:
            InputMap.action_add_event( action_b, e )
        pass
```

L'approccio di rimappare l'associazione tasto-azione a livello di `InputMap`, piuttosto che invertire il segno di input direttamente nel codice legato al movimento ha il vantaggio di essere quanto più trasparente possibile al resto del codice, infatti qualunque script richiami `Input.get_axis("left", "right")` riceve in automatico il valore invertito, senza il bisogno che venga modificato nulla.

Alla fine, l'alterazione della meccanica del salto è stata realizzata tramite la variabile `psychedelic_gravity_scale`, che viene moltiplicata direttamente nel calcolo della velocità verticale in `_physics_process()`:

LISTING 19: Applicazione del fattore di gravità alterata

```
    func _physics_process( _delta: float ) -> void:
        velocity.y += gravity * _delta * gravity_multiplier *
    psychedelic_gravity_scale
        velocity.y = clampf(velocity.y, -1000.0, max_fall_velocity)
        move_and_slide()
        change_state(current_state.physics_process(_delta))
        pass
```

Un valore di 0.55 abbassa la gravità effettiva a poco di più della metà di quella standard, ricreando la sensazione di "leggerezza" quando un giocatore salta o cade, mantenendo la coerenza narrativa del mondo psichedelico.

Le seguenti due Figure 4.6 e 4.7 mostrano il confronto visivo tra le due modalità del videogioco tra la modalità di gioco ordinaria e quella psichedelica:

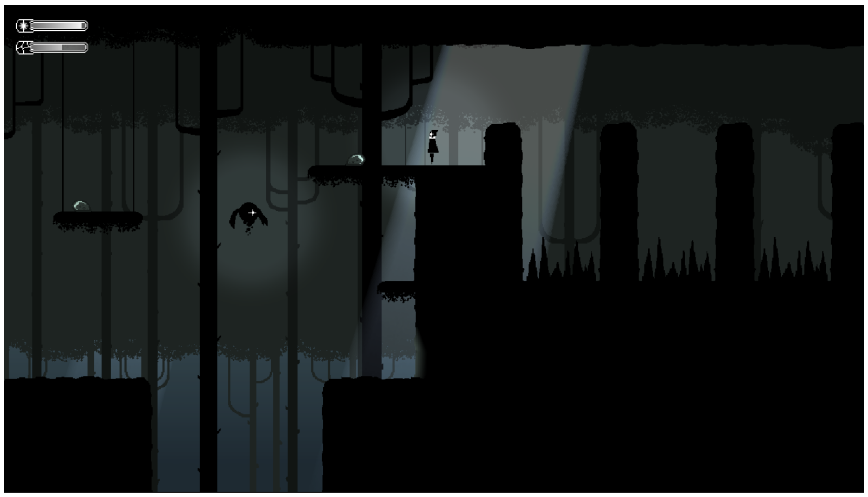


FIGURA 4.6: Modalità di gioco ordinaria.



FIGURA 4.7: Modalità di gioco `"Psichedelico"`.

4.6 Flusso dei dati e strumenti utilizzati

4.6.1 Flusso dei dati end-to-end

Nelle precedenti sezioni si sono descritte le diverse componenti del sistema. In questa sottosezione si riprende il loro funzionamento nel complesso, mostrato nella seguente Figura 4.8, insieme al ciclo di ogni finestra temporale Δt = 5secondi, dalla cattura dell'input fino alla possibile attivazione della modalità "*psichedelica*".

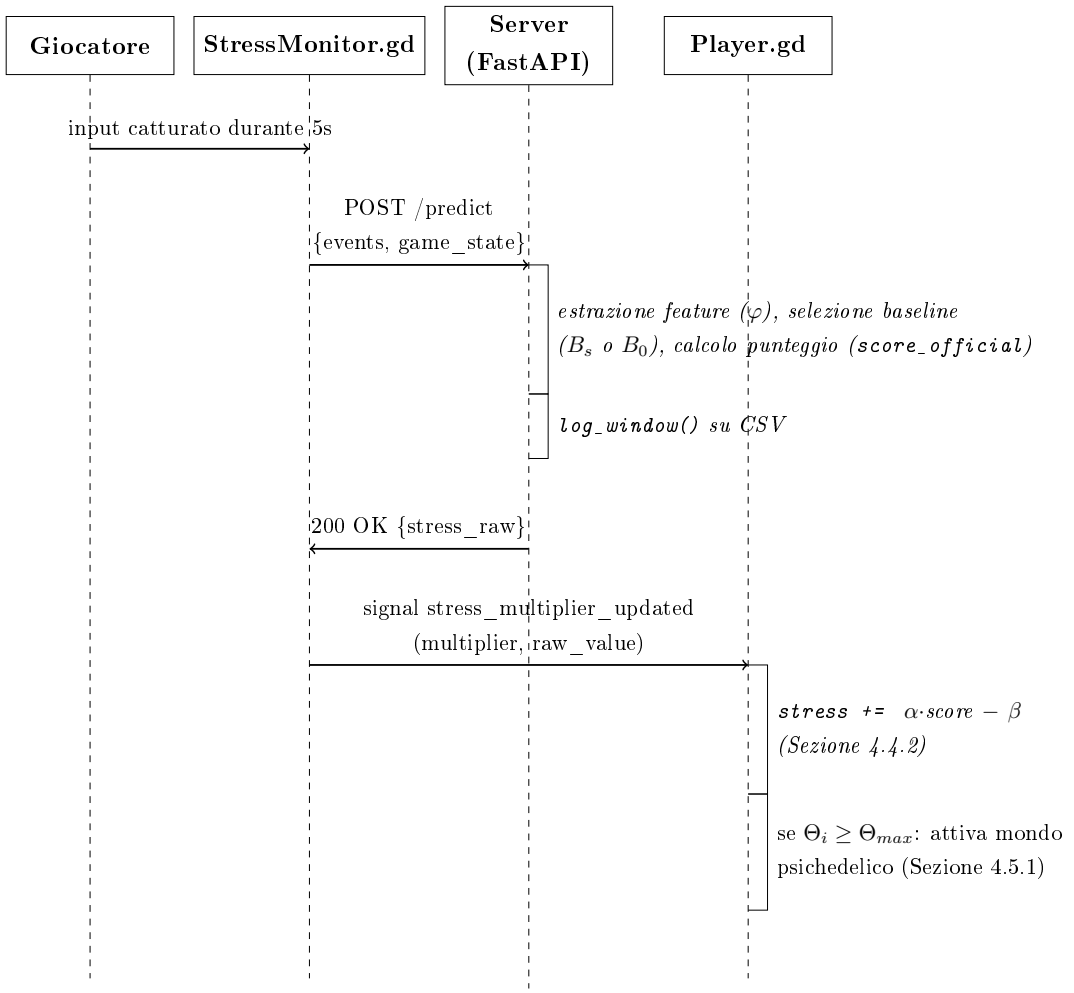


FIGURA 4.8: Diagramma di sequenza del ciclo end-to-end. L'intero flusso si ripete a ogni finestra temporale (Δt = 5s) per l'intera durata della sessione.

Come si può osservare, il diagramma mostra nel dettaglio la separazione netta tra l'elaborazione lato server (il blocco centrale, tra la ricezione della richiesta e l'invio della risposta) e l'elaborazione lato client; inoltre, si nota come lo stesso ciclo alimenti parallelamente due destinazioni diverse: il log della sessione su file, utilizzato per l'analisi del Capitolo 5, e l'aggiornamento della barra dello stress comportamentale, utilizzata per l'adattività del gameplay in tempo reale.

4.6.2 Strumenti e librerie usate

La Tabella 4.2 riassume gli strumenti e le librerie utilizzati nello sviluppo di ciascuna delle componenti del sistema, con il ruolo specifico svolto da ciascun elemento, implementato e descritto nelle sezioni precedenti:

Componente	Strumento	Ruolo nel sistema
Client	Godot 4	motore di gioco; gestione della scena, della macchina a stati del personaggio e della logica di gameplay
	`HTTPRequest`	nodo Godot nativo per l'invio delle richieste HTTP al server (Sezione 4.2.1)
	`JSON`	serializzazione/deserializzazione dei payload scambiati con il server
	`InputMap`	API di Godot per la rimappatura degli eventi input-azione, usata per l'inversione dei comandi (Sezione 4.5.1)
Server	Python 3	linguaggio di implementazione del servizio di analisi comportamentale
	FastAPI	framework web per l'esposizione degli endpoint `/predict`, `/reset`, `/log_event`
	Uvicorn	server ASGI su cui gira l'applicazione FastAPI
	Pydantic	validazione dello schema dei payload in ingresso (`WindowPayload`, `GameEvent`)
	NumPy	calcolo numerico: estrazione feature, normalizzazione, statistiche di baseline
	scikit-learn	implementazione dei quattro modelli di anomaly detection (`IsolationForest`, `OneClassSVM`, `LocalOutlierFactor`, `EllipticEnvelope`)
	`csv` (libreria standard)	scrittura dei log di sessione ed eventi su file
Analisi (post-sessione)	Matplotlib	generazione dei grafici di valutazione (Capitolo 5)
	`statistics` (libreria standard)	calcolo di medie e deviazioni standard aggregate su più sessioni

Tabella 4.2: Strumenti e librerie utilizzati nello sviluppo del sistema.

Capitolo 5

Sperimentazione

5.1 Obiettivi e domande di validazione

Nel Capitolo 1 è stata formulata la domanda di ricerca che corrisponde al "cuore" di questo lavoro di tesi: *è possibile rilevare in tempo reale deviazioni comportamentali rispetto alla baseline personale di un singolo giocatore, usando solo pattern di input al controller in modo da* ***adattare dinamicamente*** *il gameplay?*

In questo capitolo viene presentata la sperimentazione fatta per rispondere a questa domanda. Tale risposta è articolata in tre punti fondamentali validati in modo distinto, ognuno di questi punti corrisponde esattamente ad uno dei quattro obiettivi che sono stati definiti nel primo Capitolo.

Il primo punto riguarda l'affidabilità del rilevamento: il sistema **riesce a distinguere**, in modo affidabile un comportamento calmo e frenetico?

La risposta a questo quesito è data dal protocollo sperimentale calmo/frenetico introdotto nella Sezione 5.2, dove i risultati saranno presentati nella Sezione 5.4 per mezzo delle metriche di tasso di veri positivi, tasso di falsi positivi e latenza di rilevamento.

La seconda domanda è invece incentrata sulla scelta metodologica dei modelli di anomaly detection: *Isolation Forest*, utilizzato come modello ufficiale per le sue caratteristiche descritte nel Capitolo 3, si comporta realmente meglio degli altri modelli considerati (One-Class SVM, Local Outlier Factor e Elliptic Envelope) sugli stessi dati? La domanda trova risposta nella Sezione 5.5.

Infine la terza questione riguarda l'esperienza videoludica personale di un giocatore: il sistema di adattamento comportamentale riesce a produrre una percettibile differenza, e nella direzione del concetto dietro la *teoria del flow* richiamata nel Capitolo 1, rispetto ad una versione senza adattività? Per questa domanda la risposta si trova nel confronto tra le due versioni di gioco effettuata tramite il questionario GEQ, presentato nella Sezione 5.6.

In conclusione la Sezione 5.7 chiude questo capitolo discutendo in maniera congiunta i risultati di questi tre quesiti, verificando in quale misura confermano la questione principale della ricerca e dichiarando esplicitamente i limiti metodologici di questa sperimentazione.

5.2 Protocollo sperimentale

5.2.1 Protocollo calmo/frenetico

Per misurare in maniera oggettiva l'affidabilità del sistema di rilevamento comportamentale, per mezzo delle metriche definite nella Sezione 5.3, serve disporre, all'interno di ognuna delle sessioni di gioco, di due fasi delimitate: in una prima fase il giocatore gioca deliberatamente in modo calmo, e nell'altra gioca deliberatamente in un modo più frenetico. Senza questo confine esplicito non sarebbe possibile scegliere se una finestra di 5 secondi venga considerata un vero positivo, un falso positivo oppure nessuno dei due.

Il protocollo utilizzato si articola nella seguente sequenza applicata all'inizio di ognuna delle sessioni di gioco dotata della meccanica adattiva (Versione A, Sezione 5.2.2):

- **Calibrazione** (60 secondi, automatica): il giocatore inizia a giocare la sua sessione normalmente; quindi il sistema acquisisce e accumula le prime `CALIBRATION_WINDOWS = 12` finestre, utilizzate per stimare la baseline globale B_0 (Sezione 4.3.2);

- **Fase calma** (2 minuti): una volta finita la calibrazione iniziale, il giocatore, sotto indicazione dello sperimentatore, preme il tasto F5, registrando l'evento `calm_start` (Sezione 4.2.1 del Capitolo 4), poi prosegue giocando volutamente in modo "calmo" per 2 minuti;

- **Fase frenetica** (2 minuti): dopo i primi due minuti per la fase calma il giocatore preme il tasto F6, registrando l'evento `frenetic_start`, per poi proseguire, volutamente, in modo frenetico e irregolare per i successivi due minuti;

- **Gioco libero** (resto della sessione): alla fine delle due fasi da due minuti, il protocollo termina e il giocatore continua a giocare la sua sessione senza nessun'altra indicazione, giocando liberamente.

Una caratteristica implementativa importante di questo protocollo è che durante tutta la sua durata (calibrazione, fase calma e fase frenetica), la barra dello stress comportamentale, seppur visibile al giocatore, resta volutamente bassa, indipendentemente dal punteggio che sta venendo già calcolato:

LISTING 1: Attivazione ritardata della barra dopo il protocollo sperimentale

```
if (not state["marker_started"]
        and state["frenetic_start_session_time"] is not None
        and (session_time_now - state["frenetic_start_session_time"])
>= FRENETIC_PHASE_DURATION):
    state["marker_started"] = True

bar_stress_raw = stress_raw if state["marker_started"] else 1.0
```

É doveroso sottolineare che questo meccanismo lascia invariato il punteggio ufficiale `official_score` che viene calcolato e registrato all'interno dei log ad ogni finestra fin dall'inizio della sessione, senza alcuna interruzione: i dati necessari restano quindi completi anche durante il protocollo.

Quello che viene alterato resta solamente il valore di `stress_raw` che viene inviato al client per aggiornare la barra dello *stress comportamentale*. Tuttavia, la barra rimane invariata finché `marker_started` resta a `False`, mantenendo sempre il valore di 1.0. Solamente alla fine dei due minuti della fase frenetica la barra inizia a muoversi in base al reale comportamento del giocatore, dando quindi inizio alla fase di gioco libero.

Questa scelta di design è deliberata: mantenere la barra inattiva durante il protocollo garantisce che le due fasi vengano misurate in condizioni identiche e comparabili tra i vari giocatori, **senza** che l'adattamento del gameplay (e quindi il possibile raggiungimento del mondo psichedelico) possa interferire con la raccolta dei dati.

5.2.2 Raccolta dei dati e strutturazione del dataset

Il dataset raccolto per la fase di sperimentazione coinvolge un gruppo unico di 10 partecipanti, ognuno dei quali ha completato l'intero protocollo sperimentale tramite due sessioni di gioco separate:

- La prima sessione, **Versione A**: corrisponde alla versione adattiva del gioco (nonché quella discussa all'interno della tesi), che contiene il protocollo calmo/frenetico (Sezione 5.2.1);
- La seconda sessione, **Versione B**: corrisponde a quella senza la meccanica adattiva.

L'ordine è stato controbilanciato tra i partecipanti, alternando chi ha giocato per primo una versione e chi l'altra, questa scelta è stata presa anche in modo tale da limitare l'effetto di eventuali fattori di ordine (apprendimento e affaticamento) sul confronto che sarà discusso nella Sezione 5.6. Alla fine di ognuna delle due sessioni, il partecipante ha completato il questionario GEQ relativo all'esperienza di gioco appena fatta (Sezione 5.3.2).

Le dieci sessioni di gioco nella Versione A costituiscono il dataset utilizzato per la validazione del rilevamento di deviazione comportamentale (Sezione 5.4) e anche per il confronto tra i diversi modelli utilizzati di anomaly detection (Sezione 5.5). I venti questionari GEQ (dieci per ogni versione) costituiscono il dataset usato per il confronto dell'esperienza di gioco rispetto alle due versioni (Sezione 5.6).

5.3 Metriche di valutazione

5.3.1 Veri positivi, falsi positivi e latenza di rilevamento

La validazione del rilevamento comportamentale si basa sulle finestre della fase di calma e frenesia del protocollo sperimentale (Sezione 5.2.1), confrontando in ogni finestra il punteggio ufficiale $\hat{a}_i$ (Capitolo 3) con una soglia di rilevamento fissata a `DETECTION_THRESHOLD` $= 0.4$.

Nel momento in cui in una finestra della fase frenetica, il cui punteggio, supera la soglia viene conteggiata come vero positivo; mentre, una finestra della fase calma il cui punteggio supera la soglia viene conteggiata come falso positivo. Da questa base appena descritta si definiscono le due diverse metriche di tasso:

$$\text{TPR} = \frac{\text{numero di finestre frenetiche con } \hat{a}_i > 0.4}{\text{numero totale di finestre frenetiche}} \tag{5.1}$$

$$\text{FPR} = \frac{\text{numero di finestre calme con } \hat{a}_i > 0.4}{\text{numero totale di finestre calme}} \tag{5.2}$$

Il sistema riesce a riconoscere un comportamento frenetico quando viene identificato un TPR molto alto; mentre, un FPR basso indica che il sistema **non** segnala, in modo errato, un comportamento che di fatto è calmo. Le due metriche si leggono insieme, infatti: se un sistema classificasse ogni finestra come *anomala*, allora si otterrebbe un TPR perfetto; tuttavia, otterremmo anche un FPR molto alto, senza nessun tipo di utilità.

La terza metrica, ovvero la `latenza di rilevamento`, misura quanto tempo trascorre tra l'inizio di una fase frenetica e il momento in cui il punteggio supera per la prima volta la soglia:

$$\text{latenza} = t_{\text{prima finestra frenetica con } \hat{a}_i > 0.4} - t_{\text{frenetic_start}} \tag{5.3}$$

Un valore di latenza molto basso indica che il sistema reagisce molto velocemente al cambiamento di comportamento, un aspetto importante dato l'obiettivo di adattare il gameplay a run-time (Capitolo 1).

La soglia 0.4 è stata scelta empiricamente, confrontando un numero limitato di valori candidati, quindi, non tramite un'esplorazione sistematica e completa. Il criterio adottato è stato quello di cercare di massimizzare il divario tra TPR e FPR. Se fosse stata scelta una soglia più alta il divario tra i due valori sarebbe sceso, avvicinandoli. L'esplorazione più sistematica di tutto lo spazio delle diverse soglie rappresenta un ipotetico sviluppo futuro, argomento discusso nel Capitolo 6.

5.3.2 Il questionario GEQ

Per validare l'esperienza personale di ognuno dei giocatori (Sezione 5.6) si è utilizzato il *Game Experience Questionnaire* (GEQ) [8], che corrisponde ad uno strumento *psicometrico standardizzato* e molto utilizzato nella ricerca sui videogiochi per misurare l'esperienza del giocatore su più aree.

Questo questionario si compone di due moduli somministrati in ordine al termine di ogni sessione di gioco. Il *Core Module* (contenente 33 quesiti) misura l'esperienza complessiva durante il gioco, suddivisa in sette sottoscale: Competenza, Immersione sensoriale e immaginativa, Flow, Tensione/Fastidio, Sfida, Affetto negativo e Affetto positivo. Il *Post-Game Module* (contenente 17 quesiti) misura l'esperienza subito dopo la sessione di gioco, anche in questo caso, questa parte è suddivisa in quattro sottoscale aggiuntive (Esperienza positiva, Esperienza negativa, Stanchezza, Ritorno alla realtà). Ogni quesito è valutato dal giocatore su una scala di tipo *Likert* a cinque livelli, da 0 (che equivale a *per niente*) a 4 (che equivale a *estremamente*). Infine per ogni sottoscala viene calcolato un punteggio basato sulla media aritmetica dei diversi quesiti presenti in ogni sottoscale.

Inoltre, ai fini del confronto trattato nella Sezione 5.6; la presente tesi si sofferma sulle sette sottoscale che compongono il Core Module, poiché sono direttamente collegate all'esperienza di gameplay personale, quindi più pertinenti nel momento in cui si vanno a confrontare le due versioni di gioco.

Ogni partecipante (Sezione 5.2.2) ha risposto al questionario due volte, una alla fine della sessione nella Versione A e un'altra al termine della sessione per la Versione B, permettendo un confronto a misure ripetute tra le due condizioni sperimentali.

5.4 Risultati del rilevamento comportamentale

5.4.1 Evoluzione dello stress nel tempo: caso esemplificativo

Prima di trattare i risultati complessivi aggregati sulla base di tutto il dataset, è utile notare l'andamento del punteggio di anomalia comportamentale in una specifica sessione di gioco, per esemplificare concretamente il significato delle metriche utilizzate, definite nella Sezione 5.3.1. La sessione di gioco selezionata come esempio è quella che rappresenta meglio le quattro metriche (FPR, TPR, latenza e delta), rispetto ai valori medi calcolati su tutto il dataset (Sezione 5.4.2), questo a garanzia del fatto che la sessione scelta è effettivamente rappresentativa del comportamento tipico del sistema, piuttosto che un caso individuato perché più "favorevole". La Figura 5.1 mostra l'andamento completo della sessione e il marker delle due fasi del protocollo (individuabili grazie alle linee tratteggiate).

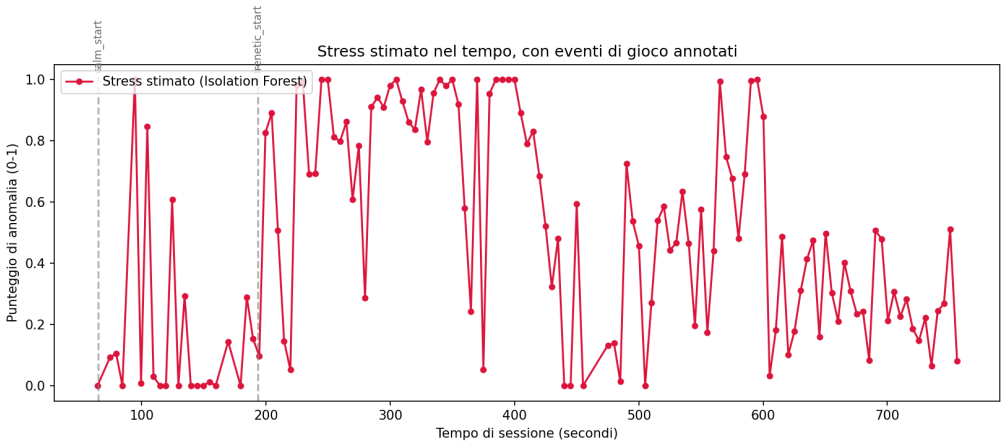


FIGURA 5.1: Andamento del punteggio di anomalia comportamentale nel tempo per una sessione rappresentativa, con le fasi calma e frenetica del protocollo sperimentale annotate.

Come si può osservare, durante la fase di calma che parte dal marker `calm_start` e termina al marker `frenetic_start`, il punteggio rimane per la maggior parte del tempo vicino al valore di zero, con soli due picchi isolati che raggiungono il valore massimo: sono proprio questi due picchi a generare il valore di *falsi positivi* pari a 14.3%, valore leggermente più alto rispetto ad altre sessioni di gioco dell'intero dataset, questo evento conferma che anche in presenza di brevi episodi come quello descritto, durante la fase di calma del protocollo, possono incidere sul tasso di falsi positivi nel momento in cui il numero delle finestre sotto osservazione è più piccolo.

Subito dopo il marker `frenetic_start`, con l'inizio della fase frenetica, il punteggio cresce e resta pressoché alto per tutta la durata di questa fase, producendo un valore di *veri positivi* pari a 83.3%.

Più avanti nella sessione dopo il termine della fase frenetica, la sessione di gioco prosegue fino a circa il secondo 760. In questa porzione finale si nota un andamento discendente e progressivo del punteggio, dove si notano singoli episodi di improvvisa risalita (ad esempio al secondo 565-600 dove il punteggio tocca il valore di 1.0): una rappresentazione della dinamica di accumulo di "stress" comportamentale con successivo decadimento costante come formalizzato nell'Equazione 3.10 del Capitolo 3, poiché in assenza di altri episodi di comportamento anomalo, il punteggio tende in modo naturale a scendere nel tempo.

5.4.2 Risultati aggregati su più sessioni

La seguente Tabella 5.1 riporta i valori di FPR, TPR, latenza di rilevamento e delta (ovvero la differenza tra il punteggio medio della fase frenetica e quello della fase calma) per ognuna delle dieci sessioni di gioco dell'intero dataset (Sezione 5.2.2), insieme alla deviazioni standard e media calcolate sull'intero campione.

Sessione	**TPR**	**FPR**	**Latenza (s)**	**Delta**
20260820_173227	66.7%	0.0%	6.25	0.462
20260821_161632	83.3%	14.3%	6.06	0.567
20260821_163543	75.0%	12.0%	20.07	0.488
20260825_104711	95.8%	8.0%	7.08	0.748
20260825_123857	95.8%	29.2%	2.30	0.598
20260825_144328	100.0%	36.8%	2.44	0.647
20260915_144630	70.8%	13.0%	6.16	0.492
20260915_154554	87.0%	8.0%	7.50	0.655
20260915_161251	70.8%	4.0%	6.50	0.514
20260916_111812	69.6%	12.0%	2.62	0.331
Media ± dev. std	**81.5% ± 12.6%**	**13.7% ± 11.2%**	**6.70 ± 5.11**	**0.550 ± 0.118**

Tabella 5.1: Risultati del protocollo calmo/frenetico per le dieci sessioni del dataset.

Come si nota il sistema riesce a rilevare in modo corretto il comportamento frenetico in una proporzione elevata di finestre (TPR = 81.5% in media), rispetto ad un tasso di falsi allarmi più contenuto (FPR = 13.7%), con una differenza media tra le due diverse metriche pari a 67.8 punti percentuali.

La latenza di rilevamento media si aggira poco più sopra i sei secondi e mezzo, al pari di una singola finestra temporale ($\Delta t = 5$ secondi), Sezione 3.2.1 oltre l'inizio della fase frenetica.

Tuttavia risulta evidente, guardando la tabella, una variabilità da non trascurare, tra le diverse sessioni: l'FPR spazia dallo 0.0% della prima sessione al 36.8% della sesta, invece la latenza dai 2.3 secondi della quinta sessione ai 20.1 secondi della terza sessione. Questa diversità è discussa approfonditamente, insieme ai suoi possibili fattori esplicativi, nella Sezione 5.7.

5.5 Confronto tra i quattro modelli di anomaly detection

5.5.1 Risultati del confronto

Mantenendo la stessa logica di fase calma e fase frenetica e applicando sempre la stessa soglia di rilevamento pari a 0.4 usate per il punteggio ufficiale (Sezione 5.3.1), ma alla normalizzazione a 3σ sulla baseline globale descritta nel Capitolo 3 (Sezione 3.3.1), si può calcolare TPR e FPR per ognuno dei quattro diversi modelli di *anomaly detection*, aggregati su tutte e dieci le sessioni del dataset. Nella seguente Tabella 5.2 sono riportati i risultati:

Modello	**TPR (media ± dev. std)**	**FPR (media ± dev. std)**
Isolation Forest	$65.5\% \pm 35.8\%$	$30.4\% \pm 27.9\%$
One-Class SVM	$62.9\% \pm 17.9\%$	$34.7\% \pm 22.6\%$
Local Outlier Factor	$44.0\% \pm 21.9\%$	$25.6\% \pm 24.1\%$
Elliptic Envelope	$72.4\% \pm 35.2\%$	$28.2\% \pm 25.3\%$

TABELLA 5.2: Confronto tra i quattro modelli di anomaly detection, formulazione a 3σ, aggregato sulle dieci sessioni del dataset.

Osservando la differenza tra TPR e FPR come indicatore complessivo di capacità discriminativa, *Elliptic Envelope* raggiunge il risultato migliore ($72.4\% - 28.2\% = 44.2$ punti percentuali), subito dietro all'*Isolation Forest* (35.1 punti), seguito dal *One-Class SVM* (28.2 punti) ed infine a chiudere il *Local Outlier Factor*, il quale mostra il divario più piccolo (18.4 punti) con un TPR corrispondente al più basso di tutti. Va evidenziato come l'elevata deviazione standard dell'*Isolation Forest* e dell'*Elliptic Envelope* (rispettivamente 35.8% e 35.2% sul TPR), indica la grande variabilità di prestazioni da una sessione all'altra, più marcata rispetto agli altri due modelli.

Un confronto rilevante è quello tra *Isolation Forest* in questa formulazione a 3σ e la stessa *Isolation Forest* nella formulazione ufficiale a 2σ con baseline per-stato (Sezione 5.4.2): 65.5% di TPR e 30.4% di FPR nella formulazione a 3σ, contro l'81.5% di TPR e il 13.7% di FPR della formulazione ufficiale. Questa differenza conferma che i dettagli introdotti nel punteggio ufficiale (ovvero la saturazione a 2σ, l'asimmetria direzionale e le baseline personalizzate per stato di gioco) realizzano concretamente un miglioramento delle prestazione piuttosto che con questa formulazione 3σ su una baseline globale utilizzata solamente per confrontare i modelli.

La Figura 5.2 di seguito mette in evidenza, solamente a titolo qualitativo, l'andamento dei quattro diversi modelli nel tempo. Tutti i modelli mostrano un effettivo incremento in corrispondenza dell'inizio della fase frenetica, ma l'ampiezza e la stabilità della curva resta diversa: *One-Class SVM* e l'*Elliptic Envelope* riescono a raggiungere e a restare per più tempo il valore massimo, a differenza di *Local Outlier Factor* che si mantiene su un intervallo più contenuto per tutta la sessione, coerentemente con la media più bassa riportata nella tabella:

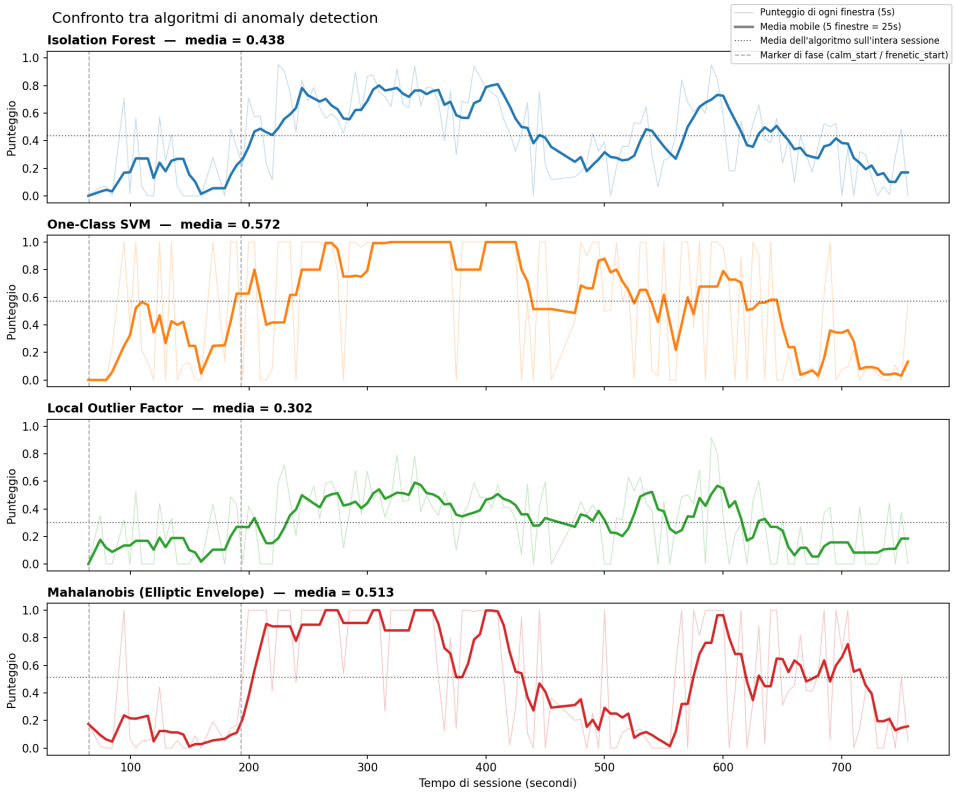


FIGURA 5.2: Confronto qualitativo tra i quattro modelli di anomaly detection per la sessione rappresentativa già presentata in Figura 5.1.

5.5.2 Feature maggiormente responsabili del rilevamento

Per ognuna delle finestre dove il punteggio ufficiale oltrepassa una soglia di riferimento, il sistema registra anche la feature con lo z-score più alto in valore assoluto (`top_contributing_feature`, Sezione 4.4.1), mostrando quindi un primo elemento di spiegabilità su quali schemi (pattern) comportamentali hanno guidato frequentemente il rilevamento. La Figura 5.3 riporta i dati rilevati dalla stessa sessione della figura precedente, la frequenza con cui ciascuna feature è risultata la più responsabile della deviazione, tra le diverse finestre con un punteggio superiore a 0.5 (dove la soglia resta specifica di questa analisi, tuttavia distinta da quella 0.4 utilizzata per le metriche di rilevamento delle sezioni precedenti).

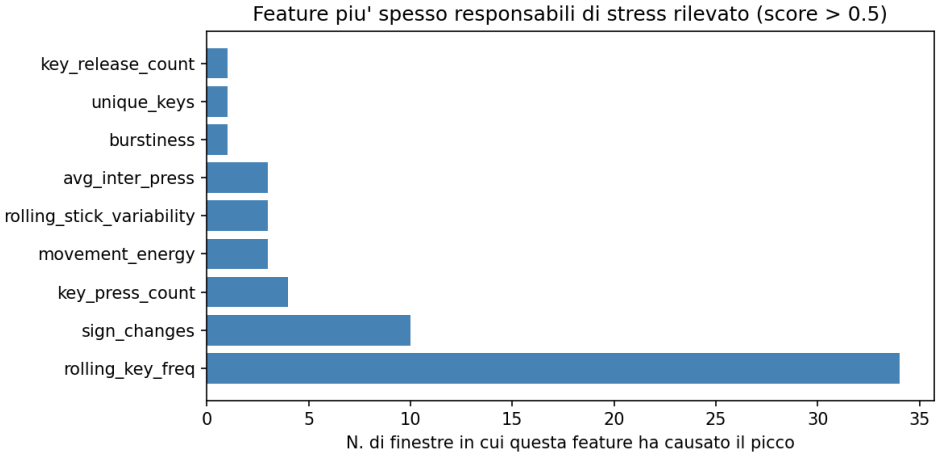


FIGURA 5.3: Frequenza con cui ciascuna feature è risultata la principale responsabile del rilevamento, per la sessione rappresentativa.

Come si può osservare, in questa specifica sessione, la feature `rolling_key_freq` (ovvero la media mobile della frequenza di pressione dei tasti sulle ultime tre finestre Sezione 4.3.1) corrisponde a quella più frequente tra tutte le altre, subito dopo si trova `sign_changes` (ovvero i cambi di direzione dello stick dell'analogico). Tutte le altre feature restano un contributo più occasionale. Questa osservazione, suggerisce che, almeno in questa sessione, il sistema tende a rilevare il comportamento più frenetico principalmente attraverso un incremento *sostenuto* nel tempo della frequenza di pressione dei tasti, piuttosto che attraverso dei singoli picchi più isolati o a pattern legati solamente al movimento dello stick.

5.6 Confronto tra la Versione adattiva (A) e quella non adattiva (B)

In questa sezione si descrive il confronto tra l'esperienza soggettiva riportata dal dataset composto dai 10 partecipanti (Sezione 5.2.2) dopo che questi ultimi hanno giocato alla Versione A (quella adattiva) e alla Versione B (quella non adattiva), misurata grazie alle sette sottoscale del *Core Module* del GEQ (Sezione 5.3.2). Considerando che il confronto è stato fatto su misure ripetute sugli stessi giocatori, per ogni sottoscala è stato applicato il test di Wilcoxon signed-rank. La seguente Tabella 5.3 riporta i risultati completi:

Sottoscala	**Media A**	**Media B**	**Differenza**	**p (Wilcoxon)**
Flow	2.68	1.76	+0.92	0.0020
Sfida	3.44	1.66	+1.78	0.0020
Tensione/Fastidio	1.53	0.30	+1.23	0.0039
Affetto positivo	3.66	3.30	+0.36	0.0078
Competenza	2.22	2.86	−0.64	0.0469
Immersione sens./immag.	3.15	2.95	+0.20	0.2188
Affetto negativo	0.12	0.05	+0.07	0.5000

Tabella 5.3: Confronto tra le sette sottoscale del GEQ Core Module, Versione A (adattiva) contro Versione B (non adattiva), $N = 10$.

Come si osserva, cinque su sette sottoscale evidenziano una differenza statisticamente significativa ($p < 0.05$) tra le due Versioni, inoltre è evidente come queste sottoscale richiamano un pattern coerente sotto più aspetti rispetto alla *teoria del flow* citata nei Capitoli 1 e 2.

Il raffronto più importante rispetto alla domanda di questa tesi è quella sulla sottoscala **Sfida**: tutti e dieci i giocatori che compongono il dataset hanno riscontrato un fattore di sfida percepita più alto nella Versione A ($p = 0.0020$) rispetto alla Versione B. Questo dato conferma, dal punto di vista soggettivo, che l'obiettivo del sistema, modificare dinamicamente il gameplay (mappa input-azione, spazio di gioco e parametri fisici) all'aumentare dello stress comportamentale, produce realmente un aumento percepito della sfida, non solo una variazione misurabile internamente al sistema.

Un'altra sottoscala che ha riscontrato un sostanziale risultato è quella del **Flow** che risulta essere più alta nella Versione A, con la stessa unanimità dei partecipanti riscontrata per la sottoscala precedente.

Questo risultato è particolarmente rilevante per il presente progetto poiché la teoria del flow prevede che l'esperienza di coinvolgimento ottimale si debba mantenere conservando l'equilibrio tra il fattore di sfida e l'abilità del giocatore (Capitolo 1); questi dati che mostrano l'aumento delle due sottoscale evidenziano come il giocatore si stia muovendo verso il proprio canale individuale di **flow**, non andando mai al di sotto di esso.

La Tensione/Fastidio risulta più alta nella Versione A, anche in questo caso ($p = 0.0039$, 9 partecipanti su 10 nella stessa direzione). Il risultato mostra qualcosa di prevedibile, dato che il sistema è costruito appositamente per far reagire un giocatore ad un aumento dello stress comportamentale. Quindi anche questo dato mostra come il sistema sia riuscito ad intensificare la sensazione di tensione.

Il dato più significativo dal punto di vista della validità complessiva di questo sistema è la sottoscala **Affetto positivo**. Oltre a risultare più alta nella Versione A ($p = 0.0078$), nonostante l'incremento della sfida e della tensione i giocatori non hanno riscontrato un'esperienza di gioco peggiore nel suo complesso, anzi piuttosto hanno riportato un miglioramento (seppur lieve). Questo risultato rafforza l'idea di un'esperienza, che nonostante la sua crescente difficoltà all'aumentare della tensione e della sfida, non è stata percepita come negativa o frustrante. Questo avvalora ancora di più l'efficacia dell'implementazione di meccaniche adattive all'interno di un videogioco.

Il penultimo risultato che si può considerare più "sfumato" riguarda la sottoscala della **Competenza**, effettivamente più bassa nella Versione A ($p = 0.0469$, al limite della soglia convenzionale di significatività). Anche in questo caso il risultato mantiene coerenza con la teoria del flow: l'avvicinarsi al margine superiore del canale di flow comporta tendenzialmente una più bassa sensazione di piena padronanza a differenza di una condizione in cui la sfida resta costante e sotto le capacità di un giocatore, come nella Versione B senza meccanica adattiva.

In conclusione, l'ultima sottoscala **Immersione sensoriale e immaginativa** non riporta grandi differenze significative tra le due diverse versioni ($p = 0.2188$). Anche questo dato risulta avere un valore atteso: entrambe le versioni condividono lo stesso ambiente grafico, lo stesso tipo di narrazione e lo stesso level design di base, l'unica differenza è data dalla presenza delle meccaniche adattive e del "mondo psichedelico". In altre parole, non c'è un rilevante motivo per aspettarsi differenze sostanziali se non questi aspetti che sono condivisi.

5.7 Discussione dei risultati e limiti

Riassunto in relazione alla domanda di ricerca

I risultati raccolti e illustrati in questo capitolo permettono di rispondere, in parte, alla domanda di ricerca. Il sistema rileva effettivamente un comportamento frenetico con un tasso di veri positivi pari all'81.5% a fronte di un tasso di falsi positivi del 13.7% (Sezione 5.4.2), lo stesso avviene per la latenza media, poco oltre una singola finestra temporale. Il modello individuato come principale, nella sua formulazione completa (baseline per-stato, saturazione a 2σ, asimmetria direzionale), supera di molto la stessa *Isolation Forest* valutata con la formulazione semplificata utilizzata per il confronto tra i vari modelli (Sezione 5.5.1), confermando che le decisioni metodologiche discusse nel Capitolo 3 non sono solamente delle giustificazioni astratte, ma sono riuscite a produrre un concreto miglioramento misurabile.

Il confronto effettuato tra le due versioni ovvero quella adattiva e quella non adattiva (Sezione 5.6) fornisce una validazione indipendente, dal punto di vista dell'esperienza soggettiva di un giocatore, coerentemente con quella comportamentale: la Versione A ha prodotto più sfida percepita e più flow, **senza** una corrispondenza con l'aumento dell'affetto negativo. Prendendoli insieme, questi risultati supportano l'ipotesi di questa tesi; infatti, è possibile rilevare in tempo reale le deviazioni comportamentali personali a partire dai soli pattern di input, utilizzando questi rilevamenti per produrre un'esperienza di gioco (percepita dai partecipanti) più coinvolgente non solo più difficile.

Limiti legati alla dimensione del campione

Il dataset raccolto, composto da dieci persone, è adeguato per un lavoro di tesi triennale come questo, tuttavia rimane di dimensione modesta rispetto agli standard di altri studi sperimentali più grandi all'interno della letteratura. I risultati ottenuti andrebbero letti come **indicazioni** solide ma non definitive.

Una diretta conseguenza di questa dimensione ridotta del campione di studio può essere vista anche all'interno di ognuna delle sessioni: ciascuna fase appartenente al protocollo è composta da sole 21-25 finestre (Sezione 5.4.2), questo significa che anche una sola finestra classificata in modo diverso sposta i valori di FPR e TPR. Questo aspetto mostra la variabilità osservata tra le sessioni dove anche una sola pressione di un tasto, fatta accidentalmente durante la fase di calma, può alterare sensibilmente le percentuali misurate.

Un dataset più grande di partecipanti, con più finestre da osservare per ogni fase, ridurrebbe questo limite di sensibilità.

Altri limiti metodologici

La soglia di rilevamento utilizzata (0.4) è stata scelta in modo **empirico** confrontando il numero, limitato, di valori candidati (Sezione 5.3.1), quindi non tramite un'esplorazione sistematica dell'intero spazio delle soglie possibili. Il confronto tra i diversi modelli utilizzati di *anomaly detection* (Sezione 5.5) utilizza una formulazione più semplice e uniforme per tutti e quattro i modelli. I risultati ottenuti vanno compresi come un confronto equo *a parità di formulazione*, non come una valutazione del potenziale miglior modello. Infine, un'altra considerazione importante è quella legata alla somministrazione delle versioni del gioco, infatti seppur siano state controbilanciate tra i partecipanti non sono stati considerati fattori come la stanchezza o l'abitudine tra una sessione e l'altra.

Questi limiti, nella loro totalità, non compromettono la validità delle conclusioni raggiunte in questo capitolo, ma definiscono con chiarezza le direzioni lungo cui la sperimentazione potrebbe essere estesa e rafforzata. Alcune di queste direzioni sono riprese nel Capitolo successivo tra gli sviluppi futuri.

Capitolo 6

Conclusioni

6.1 Riepilogo del lavoro svolto e valutazione degli obiettivi

Nel Capitolo 1 si è articolato il lavoro svolto in questa tesi in quattro obiettivi. In questa sezione si ripercorrono i vari macro obiettivi, verificando in che misura si sono raggiunti, richiamando dove se ne è data dimostrazione.

Il primo obiettivo corrisponde alla **costruzione di un sistema end-to-end**, integrando un lato client sviluppato con l'engine Godot 4 con un sistema di analisi comportamentale sviluppato in Python, è stato raggiunto: nel Capitolo 3 è stato formalizzato il funzionamento concettuale, poi nel Capitolo 4 si è descritta l'implementazione completa, dall'acquisizione dell'input fino alla trasformazione del gameplay al superamento della soglia di stress comportamentale. Si è riuscito a costruire un sistema che non è rimasto solamente un prototipo isolato: le dieci sessioni di gioco raccolte per la sperimentazione (Capitolo 5) confermano che tutto il ciclo (cattura dell'input, analisi, adattamento) riesce a funzionare in modo affidabile sulle varie sessioni di gioco reali, non solo in condizioni controllate di laboratorio.

Il secondo obiettivo, ovvero la **definizione di un protocollo di validazione**, è stato raggiunto utilizzando il protocollo di fase calma/fase frenetica descritto nella Sezione 5.2.1. Le metriche che sono state ottenute (Sezione 5.4.2) mostrano un tasso di veri positivi dell'81.5% a fronte di un tasso di falsi positivi del 13.7%, con una latenza di rilevamento media contenuta: risultati che supportano l'affidabilità del sistema, nonostante ci siano dei limiti connessi alla dimensione del campione discussi nella Sezione 5.7.

Il terzo obiettivo equivale al **confronto tra i quattro modelli di anomaly detection**, anch'esso è stato raggiunto nella Sezione 5.5. L'Isolation Forest, utilizzato come modello ufficiale, si è dimostrato competitivo rispetto agli altri modelli scelti; ancora più significativo è stato il confronto che ha mostrato come gli affinamenti metodologici introdotti nel Capitolo 3 (baseline per-stato, saturazione a 2σ e asimmetria direzionale) producono un miglioramento reale e misurabile delle prestazioni, non solamente teorico.

Il quarto e ultimo obiettivo è la **validazione dell'esperienza di gioco** per mezzo del questionario GEQ, che è stato raggiunto nella Sezione 5.6. Il confronto tra la Versione A (adattiva) e quella B (non adattiva) ha mostrato un sostanziale incremento, dal punto di vista soggettivo, sia della sottoscala sfida percepita sia della sottoscala flow, senza una corrispondente crescita dell'affetto negativo (una conferma indipendente, dal lato dell'esperienza del giocatore, di quanto osservato dal lato comportamentale).

Nel loro insieme, questi quattro obiettivi e i relativi risultati permettono di affermare che tutti i punti di interesse definiti all'inizio di questa tesi sono stati raggiunti, e che la domanda di ricerca posta nel Capitolo 1 ha trovato una risposta positiva, per quanto con i limiti metodologici già discussi nella Sezione 5.7.

6.2 Conclusioni

Nel Capitolo 1 si era posta una distinzione precisa alla base di questo lavoro: la maggior parte dei videogiochi che si dichiarano "adattivi" osserva **cosa** un giocatore sceglie (una decisione narrativa, una particolare missione oppure una risposta data ad un NPC) non **come** quel giocatore si sta comportando nel momento in cui gioca. Il lavoro proposto in questa tesi ha cercato di colmare questo spazio: costruire, quindi, un sistema in grado di osservare il **come**, in modo continuo e non invasivo, e di tradurre quell'osservazione in un reale adattamento del gameplay, in tempo reale.

I risultati raccolti nel Capitolo 5 mostrano che questo obiettivo è stato raggiunto concretamente e non solamente sulla carta. Il sistema riesce a rilevare in modo affidabile le deviazioni comportamentali di un singolo giocatore rispetto al proprio standard personale, utilizzando esclusivamente i pattern di input al controller (senza l'uso di sensori biometrici, senza questionari durante la sessione e senza alcun tipo di interruzione dell'esperienza di gioco).

La cosa ancora più rilevante è che il rilevamento non resta fine a se stesso: l'incremento di sfida percepita e di flow riportato nei questionari GEQ (Sezione 5.6) che mostra come l'adattamento prodotto dal sistema viene realmente percepito dal giocatore, e viene percepito nella direzione attesa dalla teoria del flow citata dall'inizio della tesi.

Questo lavoro non pretende di risolvere definitivamente il problema della staticità della difficoltà nei videogiochi, né di sostituirsi agli approcci già consolidati in letteratura (Capitolo 2). Piuttosto rappresenta, in modo più modesto, una prova di fattibilità: dimostra, infatti, che è possibile costruire un sistema di questo tipo usando strumenti accessibili (un engine di sviluppo di videogiochi open-source, una libreria di *machine learning standard* e nessuna strumentazione dedicata come dei sensori biometrici). Un tale sistema riesce a produrre un effetto misurabile sia a livello comportamentale, sia a livello di esperienza soggettiva di un giocatore. É proprio su questa base, insieme ai limiti dichiarati, che si fondano le direzioni di sviluppo futuro discusse nella sezione seguente.

6.3 Sviluppi futuri

Questo lavoro di tesi apre diverse direzioni di sviluppo, alcune di queste sono emerse come limiti dichiarati di questo lavoro nella Sezione 5.7, altre legate in modo più diretto a possibili estensioni del sistema stesso.

1. **Esplorazione sistematica delle soglie di rilevamento.** La soglia pari a 0.4 (Sezione 5.3.1) è stata selezionata in maniera empirica, confrontando un numero ristretto di valori candidati. Uno sviluppo sarebbe l'esplorazione sistematica dell'intero spazio delle soglie possibili, ipoteticamente specifica per ogni stato del videogioco, in modo tale da verificare se un valore unico e globale possa essere davvero ottimale o se soglie diverse per stati diversi possano migliorare, ancora di più, il compromesso tra TPR e FPR.

2. **Un dataset più ampio e diversificato.** Nel caso di questa tesi il dataset raccolto per la fase di sperimentazione (composto da dieci partecipanti, Sezione 5.2.2) è adeguato per una tesi triennale di questo tipo, tuttavia la sua dimensione ridotta comporta una sensibilità non trascurabile di FPR e TPR a singole finestre isolate. Un campione di giocatori più ampio, magari più diversificato per esperienza pregressa con i videogiochi, permetterebbe stime più stabili e conclusioni più solide sul confronto tra i quattro modelli di anomaly detection.

3. **Logging completo degli eventi di gameplay.** In questo lavoro, la funzione `log_game_event()` (Sezione 4.2.1) risulta generica ed è già in grado di registrare qualunque evento, ma in questa implementazione attuale viene invocata solo per marcare l'inizio delle due fasi del protocollo sperimentale. Estendere l'uso a eventi di gameplay organici (ovvero il danno subito, la morte del personaggio, l'inizio e la fine di uno scontro contro un avversario) arricchirebbe i grafici di analisi con marker reali della sessione, invece dei soli marker manuali di test, aprendo ad un'analisi più raffinata della relazione tra eventi specifici di gioco e variazioni del punteggio di anomalia.

4. **Baseline persistente tra sessioni multiple.** Nella configurazione attuale, ognuna delle diverse sessioni di gioco riesce a calibrare una nuova baseline, senza mantenere alcun tipo di memoria delle sessioni precedenti dello stesso giocatore (Sezione 3.2.2). Un giocatore che gioca diverse volte allo stesso videogioco tende naturalmente a migliorare nel tempo, e quello che in un primo momento sarebbe associato a frenesia potrebbe diventare, con la pratica, il suo comportamento abituale. Una baseline persistente si aggiornerebbe gradualmente tra una sessione e l'altra, invece che essere ricalcolata da zero ogni volta, permettendo al sistema di seguire questa evoluzione sul lungo periodo, senza però rinunciare alla sensibilità a breve termine già garantita dalla calibrazione per ogni singola sessione.

5. **Rimozione del "cancello" temporale verso una versione giocabile.** Come già affrontato in precedenza, il meccanismo che fa restare la barra dello stress comportamentale inattiva durante il protocollo di calibrazione e valutazione (Sezione 5.2.1) è stato inserito solamente per garantire misurazioni sperimentali pulite e comparabili tra loro, mentre non corrisponde ad una caratteristica pensata per restituire un'esperienza di gioco normale. Un passo naturale verso una versione del videogioco realmente giocabile sarebbe quella di progettare una variante del videogioco priva di questo "cancello" in cui il sistema si affida solamente alla calibrazione iniziale nei primi sessanta secondi, al fine di attivarsi subito senza richiedere alcun tipo di sequenza guidata da parte di uno sperimentatore.

6. **Generalizzazione ad altri generi di gioco.** Le varie categorie di stato impiegate per la calibrazione per-stato (Sezione 4.2.2) sono state pensate in modo specifico attorno alle caratteristiche di un metroidvania. Tuttavia, sarebbe interessante vedere come l'impatto delle 22 feature comportamentali mantenga un potere discriminativo uguale su generi videoludici diversi, con dinamiche di input sostanzialmente diverse (ad esempio: un platform puro, un gioco di corse automobilistiche, uno sparatutto in prima persona, ecc.). Questo rappresenta un test naturale della generalizzabilità dell'approccio proposto, al di là del singolo caso di studio presentato in questa tesi.

7. **La spiegabilità come strumento di design, non solo di analisi.** Il campo `top_contributing_feature` (Sezione 5.5.2) è stato impiegato in questa tesi solamente a scopo di analisi a posteriori dei dati che sono stati raccolti. La stessa informazione potrebbe essere esposta in tempo reale durante le sessioni di playtesting, tramite magari un pannello che restituisce agli sviluppatori quali tra le feature comportamentali stanno guidando lo stress comportamentale rilevato in un determinato momento. Uno strumento del genere potrebbe essere utile non solo per la validazione del sistema ma anche per bilanciare il level design sulla base del comportamento osservato.

Bibliografia

[1] Alexander Baldwin, Daniel Johnson e Peta Wyeth. «Crowd-pleaser: Player perspectives of multiplayer dynamic difficulty adjustment in video games». In: *Proceedings of the 2016 Annual Symposium on Computer-Human Interaction in Play*. 2016, pp. 326–337.

[2] Markus M Breunig et al. «LOF: identifying density-based local outliers». In: *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 2000, pp. 93–104.

[3] Guillaume Chanel et al. «Emotion assessment from physiological signals for adaptation of game difficulty». In: *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans* 41.6 (2011), pp. 1052–1063.

[4] Ben Cowley et al. «Toward an understanding of flow in video games». In: *Computers in Entertainment (CIE)* 6.2 (2008), pp. 1–27.

[5] Mihaly Csikszentmihalyi e Mihaly Csikszentmihalyi. *Flow: The psychology of optimal experience*. Vol. 1990. Harper & Row New York, 1990.

[6] Deepan Das et al. «Classification and quantitative estimation of cognitive stress from in-game keystroke analysis using EEG and GSR». In: *2017 IEEE Life Sciences Conference (LSC)*. IEEE. 2017, pp. 286–291.

[7] Robin Hunicke. «The case for dynamic difficulty adjustment in games». In: *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in computer entertainment technology*. 2005, pp. 429–433.

[8] Wijnand A IJsselsteijn, Yvonne AW De Kort e Karolien Poels. «The game experience questionnaire». In: (2013).

[9] Fei Tony Liu, Kai Ming Ting e Zhi-Hua Zhou. «Isolation forest». In: *2008 eighth ieee international conference on data mining*. Ieee. 2008, pp. 413–422.

[10] Pramila Rani, Nilanjan Sarkar e Changchun Liu. «Maintaining optimal challenge in computer games through real-time physiological feedback». In: *Proceedings of the 11th international conference on human computer interaction*. Vol. 58. 2005, pp. 22–27.

[11] Peter J Rousseeuw e Katrien Van Driessen. «A fast algorithm for the minimum covariance determinant estimator». In: *Technometrics* 41.3 (1999), pp. 212–223.

[12] Bernhard Schölkopf et al. «Estimating the support of a high-dimensional distribution». In: *Neural computation* 13.7 (2001), pp. 1443–1471.

[13] Naeem Seliya, Azadeh Abdollah Zadeh e Taghi M Khoshgoftaar. «A literature review on one-class classification and its potential applications in big data». In: *Journal of Big Data* 8.1 (2021), p. 122.

[14] Pieter Spronck, Ida Sprinkhuizen-Kuyper e Eric Postma. «Difficulty scaling of game AI». In: *Proceedings of the 5th International Conference on Intelligent Games and Simulation (GAME-on 2004)*. 2004, pp. 33–37.

[15] Li Sun et al. «Detecting anomalous user behavior using an extended isolation forest algorithm: an enterprise case study». In: *arXiv preprint arXiv:1609.06676* (2016).

[16] Penelope Sweetser e Peta Wyeth. «GameFlow: a model for evaluating player enjoyment in games». In: *Computers in Entertainment (CIE)* 3.3 (2005), pp. 3–3.

[17] Chamila Wijayarathna e Erandi Lakshika. «Toward stress detection during gameplay: A survey». In: *IEEE Transactions on Games* 15.4 (2022), pp. 549–565.

[18] Georgios N Yannakakis e Julian Togelius. «Experience-driven procedural content generation». In: *IEEE Transactions on Affective Computing* 2.3 (2011), pp. 147–161.

[19] Georgios N. Yannakakis et al. «Player Modeling». In: *Artificial and Computational Intelligence in Games*. Vol. 6. Dagstuhl Follow-Ups. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2013, pp. 45–59.

[20] Philippe Zimmermann et al. «Affective computing—a rationale for measuring mood with mouse and keyboard». In: *International journal of occupational safety and ergonomics* 9.4 (2003), pp. 539–551.

[21] Mohammad Zohaib. «Dynamic difficulty adjustment (DDA) in computer games: A review». In: *Advances in Human-Computer Interaction* 2018.1 (2018), p. 5681652.

Ringraziamenti

Ringraziare qualcuno non è mai semplice, è per questo che il momento in cui ho pensato a cosa avrei scritto mi son detto che sarebbe stato il momento più difficile. Nessuna dedica scritta riuscirebbe realmente a descrivere tutto ciò che le persone mi hanno dato durante questo percorso e quanto abbiano contato e contano tutt'oggi nella mia vita, ma ci proverò lo stesso.

In primo luogo, anche se potrebbe sembrare una forma di egocentrismo, vorrei ringraziare me stesso. Prima di intraprendere il percorso universitario ho cambiato più volte la mia strada e ogni volta che ho deciso di ricominciare non mi sentivo adeguato e non sentivo un'appartenenza a nulla, ovunque mi trovassi. Nonostante ciò, con tutti i dubbi e le incertezze del caso decisi di iscrivermi a questa facoltà e di imbarcarmi in un nuovo percorso di vita maturando sempre di più che quella sarebbe stata la strada che mi avrebbe avvicinato a realizzare davvero ciò che voglio fare nella mia vita.

É per questo che dopo essere arrivato alla fine di questa strada mi sento orgoglioso di me stesso.

Un altro grazie va ai miei migliori amici Giammi e Andrea, sono le due persone che, dopo Sara, restano le più importanti della mia vita. Ci legano tanti anni di amicizia e solo voi sapete quanto vi voglia bene e quanto siate importanti per me. Senza i nostri "caffè sfogo", i nostri "cammini di Santiago" pieni di chiacchierate, le risate, gli sproni, le consolazioni, gli abbracci e le birre probabilmente anzi sicuramente non sarei il Raffaele di oggi. Quindi vi ringrazio tantissimo perché siete sempre il mio posto sicuro nel cuore.

Ringrazio i miei genitori: Carmela e Vito, che mi siete stati vicino e grazie ai quali sono riuscito a intraprendere questo percorso di studi, vi ringrazio perché non siete mai stati una presenza passiva, anzi eravate sempre lì a spronarmi quando vedevate la mia stanchezza che prendeva il sopravvento e sempre lì a consolarmi quando vedevate la mia tristezza per qualcosa.

Ringrazio anche mia nonna Rosellina, per essere sempre stata la mia fan numero uno ogni volta che passavo un esame o riuscivo a raggiungere qualcosa durante questo percorso.

Ringrazio anche Cristina e Nicola, che durante quest'avventura sono stati per me come una seconda famiglia, sempre interessati a quello che facevo e a

come stavo.

Un grazie speciale va anche ai miei amici di università, che hanno vissuto con me questi anni, tra le aule, le sessioni d'esame, i momenti più difficili (dove ci interrogavamo se fosse quella la nostra strada) e le soddisfazioni. Vi ringrazio perché non siete più e non lo siete mai stati, dei soli colleghi, siete degli Amici veri e voglio bene ad ognuno di voi e spero vi possiate realizzare ed essere sempre felici. Sono grato di avervi incontrato perché anche senza di voi non sarei mai arrivato alla fine di questa laurea.

In ultimo, ma non per importanza, ringrazio tutti i miei amici, tutte quelle persone che mi sono state vicino strappandomi un sorriso, o regalandomi una parola di conforto in un momento difficile, per ognuno di questi momenti trascorsi insieme vi ringrazio.

Ringrazio di cuore ognuna di queste persone, grazie per non avermi mai lasciato a me stesso, grazie a tutti voi perché questo traguardo è dedicato a me, a Sara e a tutti voi. Vi voglio bene!

Poi il ringraziamento più grande e più importante va alla persona che mi sta accanto, la persona che ritengo la migliore nel mondo, colei con cui ho scelto di intraprendere l'unica strada che non abbandonerò mai. Questa persona è Sara, che non è solo la mia fidanzata, ma è la mia migliore amica e la mia compagna di vita e lo è stata ancora di più nei momenti più bui di questo percorso, quei momenti dove non mi sentivo all'altezza o dove avrei voluto mollare tutto. Lei è stata la mia forza, il motivo per continuare a studiare ad impegnarmi e terminare questo percorso. Sara ti ringrazio! Perché mi hai sempre supportato e supportato, sopportato e sopportato, ed è necessario che lo scriva due volte perché la tua vicinanza è stata per me un faro ogni volta che mi cedevano le gambe dalla fatica e dalla stanchezza tu eri lì pronta a sostenermi e a risollevarmi. Se il lavoro di questa tesi esiste, gran parte del merito è solamente tuo! Ogni volta eri pronta ed entusiasta ad ascoltare ogni mia idea folle, eri lì a consigliarmi e a suggerirmi scenari diversi, meccaniche nuove e storie fantasiose.

Mi hai visto nei miei giorni peggiori, quando non riuscivo a studiare, quando mi disperavo per un esame andato male, quando sentivo la pressione della società. Ti ringrazio perché non ti sei mai tirata indietro o non mi hai mai detto: "Oggi non posso". Per tutte le volte che avevo bisogno di una persona che mi ascoltasse, eri lì ad ascoltare materie, che probabilmente non ti interessavano nemmeno, sempre diverse, sempre più difficili da esporre, figuriamoci capire, solamente ascoltando. Ti ringrazio infinitamente per tutto, amore, non immagino nemmeno come sarebbe stato questo percorso senza la tua mano, senza il tuo aiuto, non solo materiale.

```
01110100 01101001 00100000 01100001 01101101 01101111 00100000
01110100 01100001 01101110 01110100 01101001 01110011 01110011
                  01101001 01101101 01101111
```